

本科生毕业设计（论文）

基于微服务架构的在线视频分享平台的设计 与实现

Design and Implementation of an Online Video Sharing
Platform Based on Microservices

学 院：	计算机学院
专 业：	计算机科学与技术
班 级：	07812201
学生姓名：	孙将斌
学 号：	1820221055
指导教师：	胡思康

2026 年 5 月 4 日

原创性声明

本人郑重声明：所呈交的毕业设计（论文），是本人在指导老师的指导下独立进行研究所取得的成果。除文中已经注明引用的内容外，本文不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。

特此申明。

本人签名：

日期：

年

月

日

关于使用授权的声明

本人完全了解北京理工大学有关保管、使用毕业设计（论文）的规定，其中包括：①学校有权保管、并向有关部门送交本毕业设计（论文）的原件与复印件；②学校可以采用影印、缩印或其它复制手段复制并保存本毕业设计（论文）；③学校可允许本毕业设计（论文）被查阅或借阅；④学校可以学术交流为目的，复制赠送和交换本毕业设计（论文）；⑤学校可以公布本毕业设计（论文）的全部或部分内容。

本人签名：

日期：

年

月

日

指导老师签名：

日期：

年

月

日

基于微服务架构的在线视频分享平台的设计与实现

摘 要

随着移动互联网和在线视频业务的快速发展，视频分享平台已经成为用户获取信息、表达观点和参与社区互动的重要载体。传统单体架构在面对视频投稿、文件上传、视频转码、内容审核、搜索播放和互动统计等复杂业务时，容易出现模块耦合度高、扩展困难和维护成本较高等问题。针对上述问题，本文设计并实现了一个基于微服务架构的在线视频分享平台。

系统采用前后端分离与微服务相结合的设计思路，后端基于 Spring Boot 和 Spring Cloud Alibaba 构建，按照业务职责划分为网关服务、主站业务服务、资源文件服务、互动服务、后台管理服务和公共基础模块。系统使用 MySQL 保存核心业务数据，Redis 处理验证码、登录态、缓存和临时任务状态，Elasticsearch 支持视频全文检索，RabbitMQ 承担 AI 审核任务的异步消息通信，Seata 用于支撑关键跨服务写操作的一致性控制。在视频处理方面，系统实现了视频分片上传、文件合并、FFmpeg 转码和 HLS 播放资源生成；在内容审核方面，系统设计了独立的 AI 审核服务，对视频语音、声音事件和关键帧内容进行分析，并将风险等级、风险标签和审核建议返回后台，由管理员进行人工终审。

测试结果表明，系统能够完成视频浏览、搜索播放、用户互动、视频投稿、分片上传、视频转码、AI 初审、人工审核和正式发布等核心流程，整体业务链路能够形成闭环。本文的研究与实现验证了微服务架构、异步任务处理、视频资源处理和智能审核技术在在线视频分享平台中的综合应用价值。

关键词：在线视频分享平台；微服务架构；分片上传；视频转码；智能内容审核

Design and Implementation of an Online Video Sharing Platform Based on Microservices

Abstract

With the rapid development of mobile Internet and online video services, video sharing platforms have become important carriers for users to obtain information, express opinions, and participate in community interactions. However, when traditional monolithic architecture is applied to complex business processes such as video submission, file uploading, video transcoding, content review, search, playback, and interaction statistics, it may lead to high module coupling, poor scalability, and increased maintenance cost. To address these problems, this thesis designs and implements an online video sharing platform based on microservices.

The system adopts a combination of front-end and back-end separation and microservice architecture. The back-end is built with Spring Boot and Spring Cloud Alibaba, and is divided into several modules according to business responsibilities, including the gateway service, main business service, resource service, interaction service, administration service, and common basic modules. MySQL is used to store core business data, Redis is used for verification codes, login states, cache data, and temporary task states, Elasticsearch is used for full-text video search, RabbitMQ is used for asynchronous communication of AI review tasks, and Seata is introduced to support data consistency in key cross-service write operations. For video processing, the system implements chunked uploading, file merging, FFmpeg transcoding, and HLS playback resource generation. For content review, an independent AI review service is designed to analyze video speech, sound events, and key frames, and then return risk levels, risk labels, and review suggestions to the administration system for final manual review.

The test results show that the system can complete core processes such as video browsing, search and playback, user interaction, video submission, chunked uploading, video transcoding, AI preliminary review, manual review, and final publishing. The overall business process forms a complete closed loop. The design and implementation of this thesis verify the comprehensive application value of microservice architecture, asynchronous task

processing, video resource processing, and intelligent content review technology in an on-line video sharing platform.

Key Words: Online Video Sharing Platform; Microservice Architecture; Chunked Uploading; Video Transcoding; Intelligent Content Review

目 录

摘 要	I
Abstract	II
第 1 章 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	1
1.3 研究内容与目标	2
1.4 论文结构安排	3
第 2 章 相关技术介绍	4
2.1 微服务架构与服务治理技术	4
2.1.1 微服务架构思想	4
2.1.2 服务构建与运行基础	5
2.1.3 服务注册、网关与远程调用	5
2.2 数据存储、缓存与检索技术	6
2.2.1 关系型数据库与数据访问层	6
2.2.2 Redis 缓存与临时状态管理	6
2.2.3 Elasticsearch 全文检索	7
2.3 视频上传、转码与播放技术	7
2.3.1 大文件分片上传	7
2.3.2 FFmpeg 媒体转码	7
2.3.3 HLS 流媒体播放	8
2.3.4 浏览器视频播放能力	8
2.4 前端页面开发与接口调用技术	9
2.5 异步通信与任务处理技术	9
2.5.1 异步处理机制	9
2.5.2 RabbitMQ 消息队列	9
2.5.3 任务状态与一致性控制	10
2.6 智能内容审核技术	10
2.6.1 视频内容审核思路	10
2.6.2 语音识别与声音事件识别	10
2.6.3 多模态理解与人机协同审核	11
2.7 本章小结	11

第 3 章 系统需求分析	13
3.1 系统总体需求分析	13
3.1.1 需求分析目标	13
3.1.2 系统参与角色划分	13
3.1.3 系统总体业务流程	13
3.2 用户端功能需求分析	14
3.2.1 游客访问与用户认证需求	14
3.2.2 视频浏览、搜索与播放需求	14
3.2.3 社区互动需求	15
3.3 创作中心功能需求分析	15
3.3.1 视频投稿需求	15
3.3.2 视频上传与文件处理需求	16
3.3.3 稿件状态与作品管理需求	16
3.4 管理端与 AI 审核需求分析	17
3.4.1 管理员认证与分类管理需求	17
3.4.2 人工审核与 AI 审核需求	17
3.4.3 视频状态流转需求	18
3.5 非功能需求分析	18
3.6 系统可行性分析	19
3.7 本章小结	20
第 4 章 系统总体设计	21
4.1 系统总体架构设计	21
4.1.1 总体设计思路	21
4.1.2 系统整体架构	22
4.1.3 系统分层结构	22
4.2 微服务模块划分	23
4.2.1 模块划分依据	23
4.2.2 系统服务模块说明	23
4.2.3 服务协作关系	24
4.3 网关与接口设计	25
4.3.1 网关路由设计	25
4.3.2 外部接口与内部接口划分	25
4.3.3 接口规范设计	26

4.4	事务与一致性设计	26
4.5	数据库总体设计	27
4.5.1	数据库设计原则	27
4.5.2	核心数据表设计	27
4.5.3	投稿数据与正式数据分离设计	27
4.5.4	AI 审核数据设计	28
4.6	缓存、搜索与文件存储设计	29
4.6.1	Redis 缓存与临时状态设计	29
4.6.2	视频搜索索引设计	29
4.6.3	文件存储目录设计	30
4.7	消息通信与 AI 审核任务设计	31
4.7.1	异步任务总体设计	31
4.7.2	AI 审核消息通道设计	31
4.7.3	任务状态与异常处理思路	33
4.8	核心业务流程设计	33
4.8.1	视频投稿、转码与审核流程	33
4.8.2	视频状态流转设计	34
4.8.3	发布与搜索同步设计	34
4.9	安全与异常处理设计	35
4.9.1	权限控制设计	35
4.9.2	异常场景处理设计	35
4.10	本章小结	36
第 5 章	系统详细设计与实现	38
5.1	用户端功能模块实现	38
5.1.1	用户端整体功能实现	38
5.1.2	首页浏览与搜索实现	38
5.1.3	视频播放页实现	39
5.1.4	用户认证与登录状态实现	39
5.2	创作中心与视频投稿实现	40
5.2.1	创作中心功能结构	40
5.2.2	视频分片上传实现	41
5.2.3	投稿信息保存实现	41

5.3 视频转码与资源服务实现	43
5.3.1 资源服务职责	43
5.3.2 视频转码实现流程	43
5.3.3 播放资源访问实现	43
5.4 互动功能模块实现	44
5.5 跨服务事务与一致性实现	45
5.6 后台管理功能实现	45
5.6.1 后台管理端整体实现	45
5.6.2 视频审核管理实现	46
5.6.3 分类与内容管理实现	46
5.7 AI 审核任务编排实现	47
5.7.1 AI 审核任务创建	47
5.7.2 消息队列与审核服务调用	47
5.8 多模态 AI 审核服务实现	48
5.8.1 AI 审核服务结构	48
5.8.2 文本、音频与画面审核实现	48
5.8.3 审核结果回写实现	50
5.9 AI 初审与人工终审协同实现	50
5.9.1 人机协同审核机制	50
5.9.2 审核状态流转实现	51
5.10 异常处理与安全控制实现	51
5.10.1 异常处理实现	51
5.10.2 权限与接口安全实现	52
5.11 本章小结	52
第 6 章 系统测试与结果分析	53
6.1 测试环境与测试方案	53
6.1.1 测试环境配置	53
6.1.2 测试方案设计	53
6.2 核心功能测试	55
6.2.1 用户端功能测试	55
6.2.2 创作中心功能测试	55
6.2.3 后台管理功能测试	56

6.3 视频处理与业务链路测试	56
6.3.1 视频投稿到发布链路测试	56
6.3.2 视频转码与播放资源测试	57
6.4 AI 审核功能测试	57
6.4.1 AI 审核任务流转测试	59
6.4.2 多模态审核结果测试	59
6.5 跨服务事务一致性测试	60
6.6 异常场景与安全性测试	60
6.7 测试结果分析	61
6.7.1 测试结果汇总	61
6.7.2 存在问题与改进方向	61
6.8 本章小结	62
第 7 章 总结与展望	64
7.1 研究工作总结	64
7.2 项目主要特点	64
7.3 项目不足	65
7.4 后续优化方向	66
7.5 本章小结	67
结 论	68
参考文献	70
附 录	72
附录 A L ^A T _E X 环境的安装	72
附录 B B _I Thesis 使用说明	72
致 谢	73

第1章 绪论

1.1 研究背景与意义

随着移动互联网、智能终端和网络带宽的持续发展，在线视频已经成为互联网内容传播的重要形式。用户不再只是被动观看视频，也会通过投稿、评论、弹幕、点赞、收藏、投币和关注等方式参与内容生产与社区互动。因此，一个完整的视频分享平台不仅要支持视频播放，还需要具备内容生产、资源处理、用户互动、内容检索和后台治理等能力。

在线视频平台的业务链路较长，涉及用户管理、视频投稿、文件上传、视频转码、内容审核、搜索播放和互动统计等多个环节。若所有功能集中在单体系统中，随着功能增加，容易出现模块耦合度高、维护困难和扩展不便等问题。微服务架构强调围绕业务能力拆分服务，各服务可以独立开发、部署和扩展，适合用于功能模块较多、业务边界较清晰的 Web 平台系统^[1-3]。

与此同时，用户生成内容数量不断增加，平台内容安全问题也更加突出。仅依赖人工审核会带来效率低、工作量大和响应不及时等问题。语音识别、声音事件检测和多模态模型的发展，为视频内容审核提供了新的技术手段。将 AI 初审结果提供给管理员参考，可以在保证人工终审的基础上提高审核效率，形成“AI 初审 + 人工终审”的协同审核流程^[4-6]。

基于上述背景，本文设计并实现一个基于微服务架构的在线视频分享平台。系统围绕普通用户、内容创作者和管理员三类主要对象展开，完成视频浏览、搜索、投稿、转码、互动、后台审核和 AI 辅助审核等功能。课题的实践意义在于：一方面，通过微服务拆分提高系统结构清晰度和可维护性；另一方面，通过视频处理、异步任务和智能审核等模块，构建较完整的视频平台业务闭环。

1.2 国内外研究现状

国外在线视频平台起步较早，YouTube、Netflix 等平台在视频分发、流媒体播放、内容推荐和系统架构方面积累了较多实践经验。随着云计算和微服务技术的发展，大型内容平台普遍通过服务拆分、负载均衡、弹性扩展和自动化部署支撑复杂业务。与此同时，HLS 等流媒体协议和 FFmpeg 等开源音视频工具也被广泛应用于视频转码、

切片和格式转换场景^[7-8]。

国内在线视频和短视频平台发展迅速，哔哩哔哩、抖音、快手等平台在内容创作、弹幕互动、社区运营和内容审核方面形成了较成熟的业务模式。国内企业在后端架构上也大量采用 Spring Cloud、Nacos、Redis、Elasticsearch、RabbitMQ 等技术，以应对业务模块复杂、数据读写频繁和服务扩展困难等问题^[9-13]。

在内容审核方面，国内外平台均逐渐从单纯人工审核转向“机器审核 + 人工复核”的模式。传统审核方式主要依靠关键词、规则和简单分类模型，面对复杂语义和音视频混合内容时存在局限。近年来，多模态人工智能技术能够综合分析语音文本、声音事件和画面内容，为视频审核提供更丰富的判断依据^[4,6,14]。

总体来看，在线视频平台相关技术已经较为成熟，但对于毕业设计而言，难点在于如何将微服务架构、视频处理、用户互动、搜索功能和 AI 审核服务整合为一个可运行的系统。本文在现有技术基础上，构建一个具备核心业务闭环的视频分享平台原型，以体现相关技术在复杂 Web 系统中的综合应用价值。

1.3 研究内容与目标

本文围绕“基于微服务架构的在线视频分享平台设计与实现”展开，重点研究如何将视频平台中的用户管理、视频投稿、资源处理、互动行为、后台审核和 AI 内容审核等功能进行合理拆分与协同实现。系统采用前后端分离架构，后端以 Spring Cloud Alibaba 微服务体系为基础，AI 审核部分采用独立 Python 服务实现。

本文主要研究内容如下：

1. 完成在线视频分享平台的需求分析与总体设计。根据平台业务特点，明确普通用户、内容创作者、管理员和 AI 审核服务的功能边界，并设计系统总体架构和服务模块划分。
2. 实现视频投稿、上传、转码和播放链路。系统支持创作者上传视频和封面，保存投稿信息，并通过资源服务完成视频合并、转码和 HLS 播放资源生成^[7-8]。
3. 实现视频浏览、搜索和互动功能。用户可以浏览公开视频、按关键词检索视频、进入详情页播放视频，并进行评论、弹幕、点赞、收藏、投币和关注等互动操作。

4. 实现后台管理与人工审核功能。管理员可以维护视频分类、查看待审核稿件、查看 AI 审核结果，并对视频进行通过或驳回处理。
5. 实现独立 AI 多模态审核服务。AI 服务通过消息队列接收审核任务，对视频语音、声音事件和关键帧内容进行分析，并将风险等级、风险标签和审核建议返回后端^[4,6,13,15]。

本文的总体目标是完成一个功能相对完整、结构清晰、流程闭环的在线视频分享平台原型。系统应能够体现微服务架构在复杂业务拆分中的应用，能够完成视频从上传、处理、审核到发布播放的主要流程，并能够通过 AI 审核服务提高内容审核的辅助决策能力。

1.4 论文结构安排

本文共分为七章，各章内容安排如下：

表 1-1 论文章节结构安排

章节	主要内容
第一章	介绍课题研究背景、研究现状、研究内容与论文结构。
第二章	介绍系统开发涉及的关键技术，包括微服务、数据存储、视频处理、消息队列和 AI 审核技术。
第三章	从角色、功能、非功能需求和可行性等方面分析系统需求。
第四章	说明系统总体架构、服务划分、数据库、缓存搜索、文件存储和核心流程设计。
第五章	介绍用户端、创作中心、资源处理、后台管理和 AI 审核等模块的详细实现。
第六章	对系统核心功能、视频处理流程、AI 审核流程和异常场景进行测试与结果分析。
第七章	总结本文完成的工作，分析项目不足，并提出后续优化方向。

第2章 相关技术介绍

在线视频分享平台的实现需要多个技术环节协同完成。系统既要处理用户、视频、评论、审核等结构化业务数据，也要支持大文件上传、视频转码、流媒体播放、全文检索、异步任务和智能内容审核等能力。因此，本章围绕论文后续设计与实现中直接使用的关键技术进行说明。

与系统总体设计和详细实现章节不同，本章重点介绍相关技术的作用、适用场景和选型依据，不展开具体接口、数据库表和业务流程细节。这样可以避免与第四章、第五章内容重复，并使相关技术介绍更加聚焦。

2.1 微服务架构与服务治理技术

2.1.1 微服务架构思想

在功能较少的系统中，单体架构能够降低开发和部署成本。但在线视频分享平台涉及用户认证、视频投稿、资源转码、搜索检索、互动评论、后台审核和内容安全等多个业务领域。如果所有功能都集中在一个应用中，代码耦合度会逐渐升高，后续扩展和维护也会变得困难。

微服务架构强调围绕业务能力拆分服务，每个服务负责相对独立的职责，并通过轻量级通信方式进行协作^[1]。对于视频平台而言，视频资源处理、普通业务管理和智能审核任务在数据形态、处理耗时和访问压力上存在明显差异，采用服务化拆分有助于降低模块之间的耦合。

服务拆分并不是简单地按照代码目录划分，而是需要结合业务职责、数据边界和访问压力进行分析。表 2-1 对服务拆分时应关注的因素进行了概括。

表 2-1 服务拆分原则概括

原则	说明
职责清晰	每个服务围绕明确业务能力设计，避免一个服务承担过多无关功能
高内聚低耦合	服务内部功能联系紧密，服务之间减少不必要的直接依赖
数据边界明确	尽量让核心数据归属于对应服务，降低跨服务频繁读写带来的复杂度
便于扩展	对访问压力大或处理耗时的模块，可单独扩展和部署
运维可控	服务数量需要与项目规模匹配，避免过度拆分导致管理成本过高

2.1.2 服务构建与运行基础

Spring Boot 是 Java Web 开发中常用的服务构建框架。它通过自动配置、起步依赖和内嵌服务器等机制减少重复配置，使开发者能够较快搭建独立运行的后端服务^[16]。在本类系统中，用户服务、资源服务、互动服务和后台管理服务都可以基于 Spring Boot 构建。

当系统拆分为多个服务后，还需要处理服务注册、配置管理、网关路由、远程调用等问题。Spring Cloud 提供了一套微服务开发组件，用于支持分布式系统中的服务治理能力^[9]。因此，Spring Boot 更侧重单个服务的快速构建，Spring Cloud 更侧重多个服务之间的协同管理。

2.1.3 服务注册、网关与远程调用

在微服务环境中，服务实例可能因为重启、扩容或部署迁移而改变地址。如果调用方直接写死服务地址，会降低系统的灵活性和可维护性。服务注册与发现机制可以让服务启动后向注册中心登记自身信息，调用方再通过服务名获取可用实例。Nacos 支持服务发现和配置管理，适合用于微服务系统中的注册中心和配置中心^[10]。

系统对外访问通常还需要统一入口。Spring Cloud Gateway 是 Spring Cloud 体系中的网关组件，可以根据路由规则将请求转发到不同服务，并在网关层统一处理跨域、鉴权、日志和内部接口隔离等通用逻辑^[17]。网关的引入可以减少客户端直接感知多个内部服务的复杂度。

服务之间的同步调用可以使用 OpenFeign 实现。OpenFeign 是声明式 HTTP 客户端，开发者通过接口描述远程调用关系，由框架完成请求发送与响应转换^[18]。但同步调用适合处理耗时较短、需要立即返回结果的场景，视频转码和 AI 审核等长耗时任务更适合使用异步消息机制。

在跨服务写操作场景中，单个服务的本地事务难以覆盖完整业务链路。Seata 是面向微服务架构的分布式事务解决方案，能够通过全局事务协调机制处理跨服务数据提交与回滚问题^[19]。本系统在公共基础模块中引入 Seata，并在涉及投稿审核、互动计数和用户资产变更等关键写操作时，为数据一致性控制提供技术支撑。

表 2-2 微服务治理相关技术作用

技术	主要作用	适用环节
Spring Boot	快速构建独立运行的后端服务	各业务服务基础框架
Spring Cloud	提供微服务治理组件	多服务协同与治理
Nacos	服务注册发现、配置管理	服务地址管理与配置集中维护
Spring Cloud Gateway	统一系统入口与请求转发	前端访问、鉴权、内部接口保护
OpenFeign	声明式远程调用	服务之间的短耗时同步调用
Seata	分布式事务协调、全局提交与回滚控制	跨服务写操作的数据一致性控制

2.2 数据存储、缓存与检索技术

2.2.1 关系型数据库与数据访问层

关系型数据库适合保存结构清晰、关系明确的业务数据。MySQL 是常用的关系型数据库管理系统，具备成熟的事务、索引和查询能力^[20]。在在线视频分享平台中，用户信息、视频基本信息、分类信息、评论记录、审核记录等数据都属于典型的结构化业务数据，适合由关系型数据库统一管理。

为了避免在业务代码中直接拼接大量 SQL，Java 后端通常会引入数据持久层框架。MyBatis 支持通过 XML 或注解编写 SQL，并将查询结果映射为 Java 对象^[21]。相较于完全自动化的 ORM 框架，MyBatis 保留了较强的 SQL 控制能力，适合查询条件较灵活的业务系统。

2.2.2 Redis 缓存与临时状态管理

缓存技术主要用于提升热点数据访问速度，减少数据库压力。Redis 是基于内存的数据存储系统，支持字符串、列表、集合、有序集合和哈希等数据结构^[11]。由于其读写速度较快、数据结构灵活，常用于验证码、登录状态、访问计数、分类缓存和临时任务状态等场景。

需要注意的是，Redis 更适合保存热点数据和临时状态，不能简单替代关系型数据库。核心业务数据仍应保存在 MySQL 中，缓存层则用于提升访问效率或辅助异步流程。这样可以让正式数据和临时数据各自承担合适职责。

2.2.3 Elasticsearch 全文检索

视频平台的搜索功能不仅需要根据标题和简介进行关键词匹配，还需要考虑分词、相关性排序和搜索结果召回等问题。普通数据库虽然可以进行模糊查询，但在文本内容较多、检索条件复杂时效率和效果都会受到限制。

Elasticsearch 是分布式搜索与分析引擎，基于倒排索引实现全文检索，常用于搜索、日志分析和复杂查询场景^[12]。在内容类平台中，Elasticsearch 通常作为业务数据库之外的检索索引层存在。MySQL 负责保存权威业务数据，Elasticsearch 负责提升搜索体验。

表 2-3 数据处理相关技术对比

技术	主要特点	适用场景
MySQL	结构化存储、事务能力成熟	用户、视频、评论、审核等核心业务数据
MyBatis	SQL 可控、便于对象映射	数据访问层封装和复杂查询组织
Redis	内存读写快、数据结构灵活	验证码、登录态、缓存、计数和临时状态
Elasticsearch	倒排索引、全文检索能力强	视频标题、简介、标签等内容检索

2.3 视频上传、转码与播放技术

2.3.1 大文件分片上传

视频文件通常体积较大，如果一次性上传完整文件，容易受到网络波动、接口超时和浏览器限制影响。分片上传的基本思想是将大文件划分为多个较小片段，客户端逐个上传，服务端记录上传进度并在所有分片到达后完成合并。

分片上传可以降低单次请求失败造成的重传成本，也便于系统记录上传状态。对于视频投稿场景而言，分片上传能够提升大文件上传的稳定性，是后续转码和审核流程能够顺利执行的基础。

2.3.2 FFmpeg 媒体转码

用户上传的视频可能来自不同设备和软件，视频格式、编码方式、分辨率和码率并不统一。如果系统直接提供原始文件播放，可能出现浏览器兼容性差、加载慢或播放失败等问题。因此，视频平台通常需要对上传文件进行统一转码。

FFmpeg 是常用的开源音视频处理工具，可以完成视频转码、格式转换、音频提取、截图和切片等操作^[7]。通过转码，系统可以将不同来源的视频转换为适合在线播放的格式，并根据需要生成封面、音频分析文件或流媒体播放资源。

媒体转码属于耗时操作，不适合放在用户上传请求中同步完成。更合理的方式是在用户提交视频后生成后台任务，由异步消费者完成转码处理，再将处理状态回写到数据库或缓存中。

2.3.3 HLS 流媒体播放

在线播放视频时，直接加载完整视频文件会增加初始等待时间，也不利于弱网环境下的播放体验。HLS 是一种基于 HTTP 的流媒体传输协议，它将视频组织为播放列表和多个媒体分片，播放器先读取播放列表，再按需加载对应片段^[8]。

HLS 常见文件包括 .m3u8 播放列表和 .ts 媒体分片。用户看到的是连续播放的视频，系统实际提供的是一组可按需加载的片段资源。这种方式有利于降低视频初始加载压力，也便于后续扩展多清晰度播放。

2.3.4 浏览器视频播放能力

现代浏览器通过 HTML5 的 video 元素提供视频播放能力，支持播放、暂停、音量控制、进度控制和全屏等基本操作^[22]。不过，完整的视频播放体验不仅依赖浏览器控件，还需要服务端提供兼容的视频格式、稳定的资源路径和合理的加载策略。

对于本系统而言，视频播放链路可以概括为“上传文件、转码处理、生成播放资源、浏览器加载播放”。各技术环节的作用如表 2-4 所示。

表 2-4 视频处理相关技术作用

技术环节	解决的问题	作用
分片上传	大文件上传失败后重传成本高	提升上传稳定性
FFmpeg 转码	原始格式不统一、播放兼容性差	生成统一播放资源
HLS 分片	完整加载视频等待时间长	支持按需加载和在线播放
HTML5 video	浏览器页面播放控制	提供前端播放能力

2.4 前端页面开发与接口调用技术

系统前端包含普通用户端和后台管理端两个部分，页面开发采用 Vue 3 与 Vite 作为基础工具链。Vue 3 负责组件化页面构建，Vite 用于提升开发启动和构建效率；Vue Router 用于管理首页、播放页、创作中心、个人中心和后台管理页面之间的路由跳转；Pinia 用于维护用户登录态、页面共享状态和部分业务状态^[23-26]。

后台管理端需要较多表格、表单、弹窗、分页和审核操作组件，因此采用 Element Plus 提供基础 UI 组件；前端与后端服务之间通过 Axios 发起 HTTP 请求，并由网关统一转发到对应微服务^[27-28]。这种前后端分离方式能够降低页面展示逻辑与后端业务逻辑之间的耦合，也便于普通用户端和管理端分别维护。

2.5 异步通信与任务处理技术

2.5.1 异步处理机制

在 Web 系统中，并不是所有操作都适合立即完成并返回结果。对于视频转码、文件清理、AI 审核等耗时任务，如果让前台请求一直等待，容易造成接口超时，也会降低用户体验。数据密集型应用设计中强调，可靠性和可维护性需要结合任务状态、故障恢复和系统解耦进行考虑^[29]。

异步处理的基本思想是：用户先提交任务，系统记录任务状态，然后由后台消费者执行具体任务。用户或管理员可以通过状态查看处理进度。这样既能减少前台请求等待时间，也能提高复杂任务处理的稳定性。

2.5.2 RabbitMQ 消息队列

RabbitMQ 是一种消息队列中间件，支持交换机、队列、路由键、消息确认、持久化和死信队列等机制^[13]。在视频平台中，RabbitMQ 可以用于连接“任务生产者”和“任务消费者”，例如上传服务产生转码任务，资源服务或转码消费者异步处理视频文件。

消息队列能够降低服务之间的直接耦合，但也会带来消息重复、消费失败、任务幂等和状态补偿等问题。因此，系统在使用消息队列时，不能只发送消息，还需要配合任务状态表、失败重试和异常记录机制。

2.5.3 任务状态与一致性控制

异步任务与同步请求的主要区别在于，异步任务通常经历待处理、处理中、处理成功、处理失败等多个状态。微服务模式中也强调，事件驱动流程往往需要配合状态记录、重试和补偿机制使用^[3]。

在视频平台中，转码任务、审核任务和文件清理任务都需要明确状态流转。任务状态既用于前端展示处理进度，也用于后台排查异常和避免重复处理。对于不要求强一致立即完成的业务，可以采用最终一致和异步补偿策略，以减少强事务带来的复杂度。

表 2-5 同步调用与异步通信对比

比较项	同步调用	异步通信
适用任务	查询、登录、短时间业务处理	转码、审核、文件处理等耗时任务
用户体验	需要等待处理结果	前台快速返回，后台继续执行
服务耦合	调用方依赖被调用方即时响应	生产者和消费者相对解耦
异常处理	失败可立即返回	需要记录状态、重试和补偿

2.6 智能内容审核技术

2.6.1 视频内容审核思路

内容审核的目标是在内容正式发布前识别可能存在的违规或风险信息。传统审核主要依赖人工判断或规则匹配，而算法内容审核通常需要与人工判断、规则体系和申诉机制结合使用^[30]。

视频内容比普通文本和图片更复杂。它同时包含画面、语音、背景声音和上下文语义，某些风险信息可能隐藏在语音内容或关键画面中。智能内容审核的作用是利用语音识别、声音事件识别和多模态理解等技术，为人工审核提供辅助依据，而不是完全替代人工判断。

2.6.2 语音识别与声音事件识别

视频中的语音内容往往包含重要语义信息。Whisper 是一种基于大规模弱监督训练的语音识别模型，在多语言语音识别任务中具有较好表现^[4]。通过语音识别，可以

将视频中的人声转换为文本，再交由规则或模型进一步分析。

除了语音内容，声音事件也可能提示视频存在风险。例如尖叫、爆炸声、警报声等异常声音，可能对应特殊场景。AudioSet 是大规模音频事件数据集，为声音事件识别研究提供了基础^[5]。YAMNet 这类声音事件分类模型关注的是“发生了什么声音”，与语音识别关注“说了什么内容”不同。

2.6.3 多模态理解与人机协同审核

多模态语义理解是指模型同时处理图像、文本、语音等多种信息，并综合判断内容含义。Qwen-VL 是视觉语言模型，能够结合图像和文本进行理解，可用于图像描述、视觉问答、文字识别和多模态推理等任务^[6]。在视频审核场景中，可以从视频中抽取关键帧，再结合语音转写文本进行综合分析。

由于智能模型可能存在误判、漏判或推理结果不稳定等问题，内容审核系统更适合采用“AI 初审 + 人工终审”的方式。AI 审核负责快速发现风险片段、生成风险等级和审核建议；管理员根据模型结果、视频内容和平台规则作出最终决定。该方式既能提高审核效率，也能保留人工判断的灵活性。

表 2-6 智能审核相关技术作用

技术	分析对象	审核作用
Whisper	视频语音内容	将语音转换为文本，辅助识别风险表达
YAMNet	背景声音和事件声音	发现异常声音事件，提供辅助线索
Qwen-VL	关键帧与文本信息	综合画面和语义，生成风险判断依据
人工审核	视频整体内容与平台规则	对 AI 结果进行复核并作出最终决策

2.7 本章小结

本章围绕在线视频分享平台中使用的关键技术进行了介绍。首先说明了微服务架构的拆分思想，以及 Spring Boot、Spring Cloud、Nacos、Gateway、OpenFeign 和 Seata 在服务治理与一致性控制中的作用；随后介绍了 MySQL、MyBatis、Redis 和 Elasticsearch 在数据存储、缓存和检索中的分工；接着说明了分片上传、FFmpeg、HLS

和 HTML5 video 在视频处理链路中的作用；之后补充了 Vue、Vite、Pinia、Vue Router、Element Plus 和 Axios 在前端页面开发中的作用；最后介绍了 RabbitMQ 异步任务处理机制，以及 Whisper、YAMNet、Qwen-VL 等技术在智能内容审核中的应用。

经过压缩后，本章仅保留与系统设计和实现直接相关的技术内容，不再展开基础概念和具体实现细节。后续章节将在这些技术基础上，进一步说明系统需求、总体设计、详细实现和测试结果。

第3章 系统需求分析

系统需求分析是系统设计与实现之前的重要环节。对于在线视频分享平台而言，需求分析不能只停留在“能够播放视频”这一层面，还需要从用户角色、业务流程、功能边界、内容审核、性能要求和系统可行性等方面进行分析。本章主要围绕系统总体需求、用户端需求、创作中心需求、管理端与 AI 审核需求、非功能需求以及可行性分析展开。

3.1 系统总体需求分析

3.1.1 需求分析目标

本课题旨在设计并实现一个基于微服务架构的在线视频分享平台。系统需要围绕视频内容的生产、处理、审核、发布和消费形成完整闭环。普通用户可以浏览、搜索、播放视频，并参与评论、弹幕、点赞、收藏、投币等互动；内容创作者可以上传视频、编辑投稿和管理个人作品；管理员可以在后台进行分类维护、视频审核和内容治理；AI 审核服务则负责对上传视频进行自动化风险识别，为人工审核提供辅助依据。

这里，补充画系统功能结构图。

从软件工程角度看，需求分析不仅要明确系统“需要做什么”，还要明确不同角色的权限边界、关键流程和质量目标。软件架构实践中也强调，系统需求应同时关注功能需求和质量属性，例如性能、安全性、可维护性和可扩展性^[31]。

3.1.2 系统参与角色划分

这一段内容删除。

系统主要参与角色包括游客、注册用户、内容创作者、管理员和 AI 审核服务。其中，游客可以浏览和搜索公开视频；注册用户在登录后可以参与互动；内容创作者负责视频投稿与作品管理；管理员负责后台管理和人工审核；AI 审核服务负责处理视频审核任务并返回辅助审核结果。角色划分如表 3-1 所示。

删除表格，应根据不同用户类型，画出对应的用例图，并给出用例说明。

3.1.3 系统总体业务流程

补充画出：业务流程图。

在线视频分享平台的核心流程可以概括为“内容生产、内容处理、内容审核、内容发布、内容消费”。视频在公开展示前需要完成上传、转码和审核，避免未经处理或未经审核的内容直接进入前台。

表 3-1 系统主要角色及功能需求

角色	主要功能需求	说明
游客	浏览视频、搜索视频、查看公开视频信息	未登录用户可进行基础访问
注册用户	登录、播放视频、评论、弹幕、点赞、收藏、投币、关注	登录后可参与平台互动
内容创作者	上传视频、编辑投稿、查看稿件状态、管理作品	负责视频内容创作与发布
管理员	后台登录、分类管理、视频审核、查看 AI 审核结果	负责平台内容管理与审核
AI 审核服务	消费审核任务、分析视频、返回审核结果	为人工审核提供辅助依据

这些概念应给出简要描述。

视频平台业务并不是简单的同步请求流程，而是由上传、转码、审核、发布和消费多个阶段组成。视频转码和 AI 审核属于耗时任务，需要通过状态流转、异步处理和结果回写保证流程可追踪。数据密集型应用设计中也强调，复杂系统需要通过状态记录和异常恢复来提升可靠性^[29]。

3.2 用户端功能需求分析

3.2.1 游客访问与用户认证需求

游客是未登录状态下访问平台的用户。平台应允许游客浏览公开视频、查看视频详情和使用搜索功能，以降低用户进入平台的门槛。但涉及用户身份绑定的操作，例如评论、弹幕、点赞、收藏、投币和关注等，应要求用户登录后才能进行。

注册用户需要支持注册、登录、退出和登录状态校验等功能。登录后，系统应能够识别当前用户身份，并根据身份允许其执行互动操作。普通用户登录态与管理员登录态应保持区分，避免后台权限被普通用户误用或越权访问。权限控制通常需要结合认证与授权机制进行设计，Spring Security 文档也对认证和访问控制能力进行了说明^[32]。

3.2.2 视频浏览、搜索与播放需求

视频浏览是用户端的基础功能。系统需要支持首页视频列表、分类筛选、视频详情展示和分 P 播放等功能。首页和分类页面应展示视频封面、标题、作者、发布时间和基础统计数据，方便用户快速了解视频内容。

搜索功能需要支持用户根据关键词查找视频。检索范围可以包括视频标题、简介和标签等字段，并支持按相关性、发布时间或热度进行排序。对于内容类平台而言，

搜索功能能够弥补单纯首页展示的不足，帮助用户主动发现目标内容。Elasticsearch 等全文检索技术可以为复杂关键词检索提供支持^[12]。

播放功能需要能够稳定加载已发布视频的播放资源。由于上传视频通常需要经过转码和切片处理，播放页应支持加载 HLS 资源，并在播放失败时给出明确提示。HLS 协议通过播放列表和媒体分片组织视频资源，适合网页端在线播放场景^[8]。

3.2.3 社区互动需求

社区互动是视频分享平台区别于普通视频播放工具的重要特点。系统需要支持评论、弹幕、点赞、收藏、投币和关注等功能。评论用于表达用户观点，弹幕与视频播放时间点相关，点赞、收藏、投币和关注用于记录用户对内容或作者的偏好。

对于互动行为，系统需要避免同一用户重复执行不合理操作，例如重复点赞或重复收藏。系统还应在用户再次进入视频详情页时展示当前互动状态，使用户能够清楚看到自己是否已点赞、收藏或关注。

用户端主要功能需求如表 3-2 所示。

表3-1要先放在3.2节的开始处，之后再开始针对每个需求进行描述。

表 3-2 用户端主要功能需求

功能类别	功能点	需求说明
基础浏览	首页视频列表、分类筛选	用户能够查看公开视频内容
搜索检索	关键词搜索、排序、热词	用户能够根据标题、简介、标签查找视频
视频播放	视频详情、分 P 切换、在线播放	用户能够稳定播放已发布视频
用户互动	评论、弹幕、点赞、收藏、投币、关注	登录用户能够参与社区互动
账号功能	注册、登录、退出、自动登录	支持用户身份识别和状态维护

3.3 创作中心功能需求分析

3.3.1 视频投稿需求

创作中心面向内容创作者，主要承担视频投稿和作品管理功能。创作者需要能够填写视频标题、选择分类、上传封面、填写标签和简介，并设置投稿相关选项。投稿功能的目标不是简单上传文件，而是将视频资源、基础信息和后续审核流程关联起来。

视频投稿应支持多分 P 或多文件场景。对于系列视频或较长视频，创作者可能需要上传多个视频片段。系统需要保存每个分 P 的标题、文件信息和排序关系，使用户在播放页可以切换分 P，管理员在审核时也可以分别查看不同文件内容。

3.3.2 视频上传与文件处理需求

视频文件通常体积较大，直接一次性上传容易受到网络波动影响。因此，系统需要支持大文件分片上传。前端将文件拆分为多个分片后逐个上传，服务端接收分片并保存临时文件，在所有分片上传完成后进行合并。

上传完成后，系统还需要对视频进行转码处理。不同设备上传的视频格式和编码方式可能不同，若不统一处理，可能导致前台播放兼容性差。FFmpeg 可用于完成视频转码、格式转换和切片等处理^[7]。从用户体验角度看，视频上传、转码和审核不应要求用户长时间等待，系统应通过稿件状态展示处理进度。

3.3.3 稿件状态与作品管理需求

创作者提交视频后，需要了解稿件当前处于什么阶段，例如转码中、转码失败、待审核、审核通过或审核不通过。状态设计应尽量清晰，避免使用用户难以理解的内部状态名称。

作品管理功能应支持查看稿件列表、查看视频状态、编辑未发布稿件、删除不需要的稿件，以及查看作品基础统计信息。作品管理页面应突出投稿时间、审核结果和处理状态，而不是只展示视频封面和播放量。

创作中心功能需求如表 3-3 所示。

表3-3要先放在3.3节的开始处，之后再开始针对每个需求进行描述。

表 3-3 创作中心功能需求

功能类别	功能点	需求说明
投稿信息	标题、分类、标签、简介、封面	保存视频基础内容信息
视频上传	分片上传、多文件上传	支持大文件稳定上传
文件处理	分片合并、视频转码、播放资源生成	使视频满足在线播放要求
状态查看	转码中、待审核、审核通过、审核失败	创作者可查看稿件处理进度
作品管理	稿件列表、编辑、删除、统计查看	创作者可管理个人投稿内容

3.4 管理端与 AI 审核需求分析

3.4.1 管理员认证与分类管理需求

管理端主要面向平台管理员。管理员需要通过后台登录后才能进入管理系统，普通用户不能直接访问后台接口。后台接口应进行身份校验，未登录或权限不足时应拒绝访问。

分类管理是后台管理中的基础功能。视频分类会影响用户投稿时的分类选择，也会影响前台视频展示、筛选和搜索。因此，系统需要支持管理员维护视频分类数据，包括新增分类、修改分类、删除分类、排序和启用停用等操作。

3.4.2 人工审核与 AI 审核需求

视频人工审核是管理端的核心需求。创作者提交视频后，视频不能直接进入正式发布状态，而需要经过转码和审核流程。管理员需要能够查看待审核视频列表，进入审核详情页，观看视频内容，查看投稿信息和 AI 审核结果，并做出通过或驳回决定。

AI 审核服务主要负责自动化风险识别。系统在视频转码完成后应创建 AI 审核任务，并将待审核视频信息发送给 AI 服务。AI 服务完成分析后，应返回审核建议、风险等级、风险标签、命中片段和原因说明等信息。AI 审核结果不直接决定视频最终发布，而是为管理员人工审核提供辅助依据。人机交互研究中也指出，AI 系统应为人工决策提供可理解、可纠错的支持，而不是完全替代人工判断^[33]。

管理端与 AI 审核相关需求如表 3-4 所示。

表3-4要先放在3.4节的开始处，之后再开始针对每个需求进行描述。

表 3-4 管理端与 AI 审核功能需求

功能类别	功能点	需求说明
管理员认证	后台登录、退出、权限校验	保证后台功能只对管理员开放
分类管理	分类新增、编辑、删除、排序	维护平台内容分类体系
视频审核	审核列表、审核详情、通过、驳回	控制视频是否正式发布
AI 审核任务	自动创建任务、投递请求、接收结果	辅助完成视频内容风险识别
AI 结果展示	风险等级、风险标签、命中片段、原因说明	帮助管理员快速定位风险内容

3.4.3 视频状态流转需求

为了保证审核流程清晰，系统中视频状态需要明确流转。投稿视频从用户提交开始，通常会经历转码中、转码失败、待审核、审核通过或审核不通过等状态。视频状态流转如图 3-1 所示。

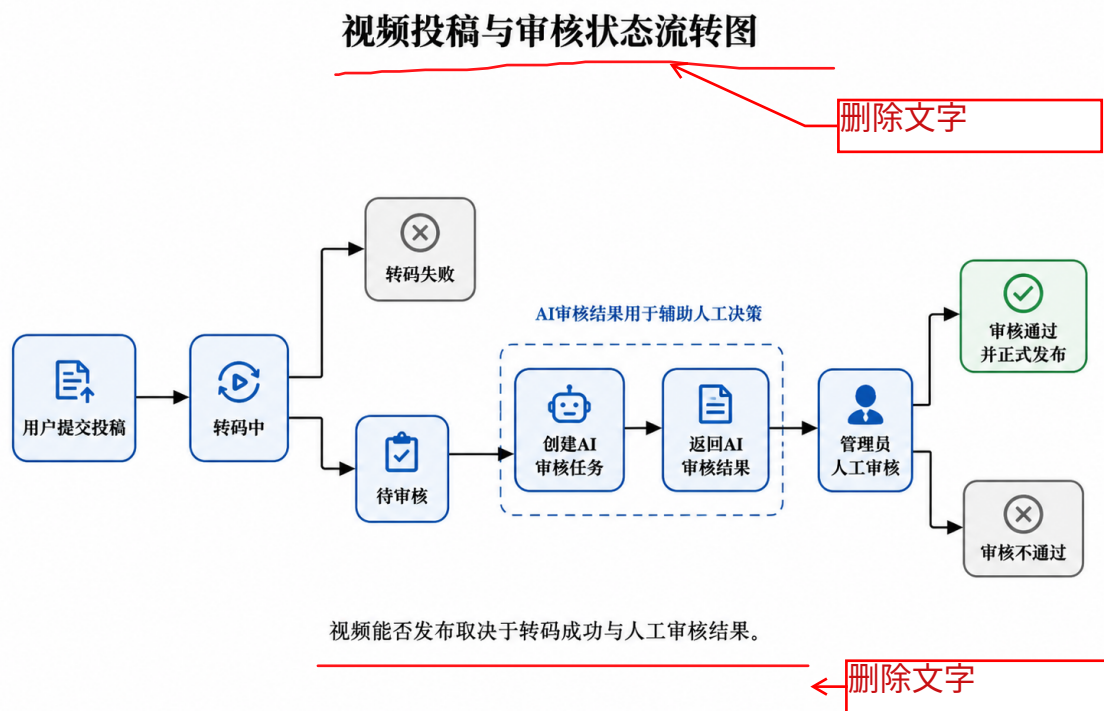


图 3-1 视频投稿与审核状态流转图

由图 3-1 可知，视频是否能够发布并不只取决于用户是否提交投稿，还取决于视频是否转码成功以及管理员是否审核通过。该状态流转规则为后续总体设计和详细实现提供了需求依据。

3.5 非功能需求分析

非功能需求主要关注系统质量属性。本系统需要在性能、可扩展性、安全性、可靠性、一致性和可维护性方面满足基本要求。

性能方面，登录、视频列表、视频详情、评论加载和搜索查询等常规接口应保

持较好的响应速度；视频上传、视频转码和 AI 审核等耗时任务应采用异步处理方式，避免用户长时间等待。ISO/IEC 25010 软件质量模型将性能效率作为软件质量的重要方面^[34]。

可扩展性方面，系统应支持后续根据业务压力扩展不同模块，例如资源处理压力较大时可以优先优化资源服务，互动访问频繁时可以优化互动模块。安全性方面，系统需要区分普通用户和管理员权限，对后台接口、上传文件和审核接口进行必要保护。

可靠性和一致性方面，系统应保证视频投稿、文件处理、审核状态和正式发布数据之间保持逻辑一致。对于涉及多个服务协作的关键写操作，系统需要具备事务协调或状态补偿能力；对于视频转码、AI 审核和播放统计等耗时或高频任务，则应通过异步处理和结果回写保证最终状态一致。若转码失败、AI 审核失败或服务短暂不可用，系统应能记录状态和失败原因，避免业务状态混乱。可维护性方面，系统应保持清晰的模块边界、接口规范和日志记录，便于后续修改和排查问题。

系统主要非功能需求如表 3-5 所示。

表 3-5 系统主要非功能需求

非功能需求	需求说明	设计关注点
性能需求	常规页面和接口响应应保持流畅	缓存、搜索、异步任务处理
可扩展性	系统应便于后续增加功能和扩展服务	模块拆分、服务边界、接口规范
安全性	区分普通用户和管理员权限，保护内部接口	身份认证、权限校验、上传限制
可靠性	异常情况下核心流程应尽量不中断	状态记录、失败原因、人工处理
数据一致性	投稿、转码、审核和发布状态应保持一致	事务控制、状态流转、结果回写
可维护性	系统结构清晰，便于后续修改和扩展	分层设计、公共模块、日志和文档

3.6 系统可行性分析

← 删除3.6节

从技术角度看，本系统涉及的关键技术均较为成熟。后端服务可以基于 Java 和微服务框架实现，数据存储可以使用关系型数据库和缓存系统，视频处理可以使用成熟的音视频工具，AI 审核也可以通过已有模型和独立服务完成。系统的主要难点

集中在视频分片上传、视频转码、审核状态流转、AI 结果回写和前后端联调等方面，但这些问题可以通过模块化设计和分阶段实现逐步解决。

从经济角度看，本系统主要基于开源技术和本地开发环境实现，不需要购买额外商业软件授权。系统开发过程中的主要成本是时间成本和硬件资源成本。对于 AI 审核部分，如果硬件条件有限，可以先使用较小模型或降低推理复杂度，以保证核心流程可运行。

从操作角度看，普通用户、内容创作者和管理员均通过 Web 页面操作。普通用户关注浏览和播放，创作者关注投稿状态，管理员关注待审核视频和 AI 风险结果。只要界面信息与角色任务相匹配，系统使用难度较低。

系统可行性分析结果如表 3-6 所示。

表 3-6 系统可行性分析汇总

可行性类别	分析内容	结论
技术可行性	主要技术成熟，系统可通过分阶段实现完成	可行
经济可行性	基于开源技术和本地环境开发，成本较低	可行
操作可行性	用户和管理员均通过 Web 页面操作，交互方式直观	可行

3.7 本章小结

本章对在线视频分享平台进行了系统需求分析。首先，从总体需求出发，明确了系统需要围绕视频内容生产、处理、审核、发布、播放和互动形成完整闭环，并分析了系统角色、业务流程和用例边界。随后，本章分别分析了用户端、创作中心、管理端与 AI 审核相关功能需求，并说明了视频从投稿到正式发布的状态流转过程。最后，本章从性能、可扩展性、安全性、可靠性、一致性和可维护性等方面分析了系统非功能需求，并从技术、经济和操作三个方面论证了系统可行性，为后续总体设计提供需求依据。

第4章 系统总体设计

删除文字

在完成系统需求分析之后，需要进一步对在线视频分享平台的整体结构、模块职责、数据组织方式和核心业务流程进行设计。总体设计的目标不是展开每一个接口或每一段代码的实现细节，而是把第三章中的功能需求转化为清晰、可维护、可扩展的系统结构，为第五章的详细设计与实现提供依据。

本章主要从系统总体架构、微服务模块划分、网关与接口边界、数据库总体设计、缓存与搜索设计、文件存储设计、消息通信与 AI 审核任务设计、核心业务流程设计以及安全与异常处理等方面展开说明。为避免与第二章相关技术介绍和第五章详细实现内容重复，本章重点说明各类技术和模块在系统整体结构中承担的设计职责。

4.1 系统总体架构设计

4.1.1 总体设计思路

4.1.1、4.1.2和4.1.3节合并，并把4.2节的系统架构图，放在这里。之后，根据系统架构图，逐一展开描述。

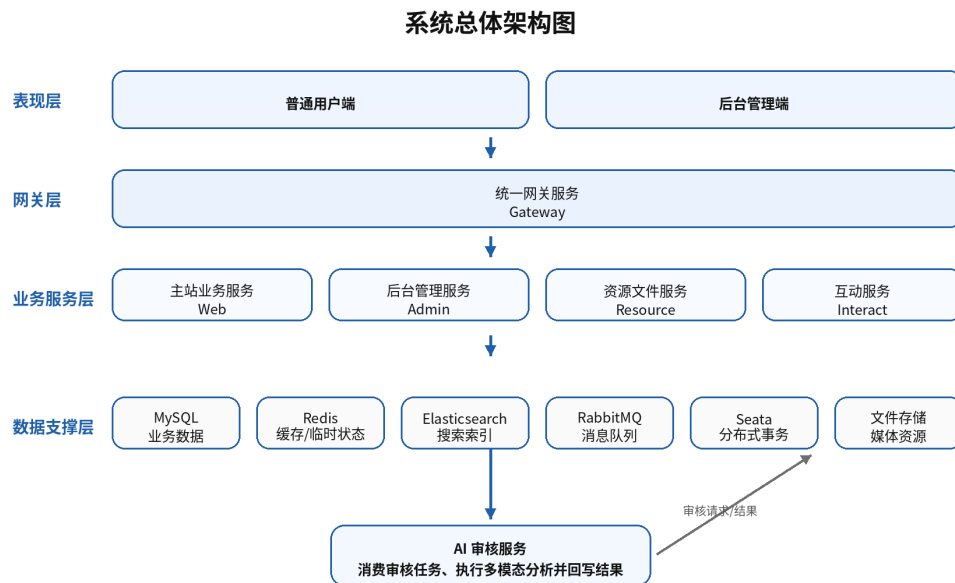
本系统采用前后端分离与微服务相结合的总体架构。前端包括普通用户端和后台管理端：普通用户端主要面向视频浏览、搜索、播放、投稿和互动等业务；后台管理端主要面向分类维护、视频审核、AI 审核结果查看和内容管理等业务。后端按照业务职责拆分为网关服务、主站业务服务、后台管理服务、资源文件服务、互动服务、公共模块和基础模块。AI 审核服务作为独立能力服务运行，通过消息队列与 Java 后端进行协作。

这种设计方式的核心思想是“前端负责交互，网关负责入口，业务服务负责领域逻辑，资源服务负责文件处理，AI 服务负责辅助审核”。如果将视频投稿、转码、互动、后台管理和 AI 分析全部放入单体应用中，系统结构会随着功能增加而变得复杂，后续维护和扩展难度也会增加。微服务架构强调围绕业务能力组织服务，能够在一定程度上降低模块间的耦合度^[1]。

从总体设计上看，系统需要满足三个要求。第一，各服务职责清楚，避免一个服务承担过多无关功能；第二，服务之间调用关系明确，避免形成混乱的循环依赖；第三，视频上传、转码、审核和发布流程能够形成完整闭环。只有满足这些条件，系统才能在后续功能扩展时保持较清晰的结构。

4.1.2 系统整体架构

系统整体架构如图 ?? 所示。外部用户通过浏览器访问普通用户端或后台管理端，请求首先进入网关服务，再由网关根据路径前缀转发到对应业务服务。业务服务之间通过服务调用进行协作，视频转码和 AI 审核等耗时任务则采用异步方式处理。



说明：网关统一接入请求，业务服务按职责拆分；数据层提供存储、缓存、搜索、消息和事务支撑。

由图 ?? 可以看出，系统不是简单的三层结构，而是在表现层、业务层和数据层基础上增加了网关入口、资源处理能力和 AI 审核能力。网关层负责统一入口和请求分发，业务服务层负责处理具体业务，数据支撑层负责保存业务数据、缓存状态、搜索索引和异步消息，AI 审核服务则负责内容风险分析。

4.1.3 系统分层结构

从架构层次上看，系统可以划分为表现层、网关层、业务服务层、AI 能力层和数据支撑层。分层设计的作用是让不同层次承担不同职责，减少功能混杂。企业应用架构模式中也强调，分层能够帮助系统保持结构清晰，并降低不同关注点之间的耦合[35]。

通过表 4-1 可以看出，系统分层不是为了形式上的复杂，而是为了明确职责边界。表现层不直接处理复杂业务，网关层不保存业务数据，AI 能力层不直接决定视

表 4-1 系统架构层次及职责

架构层次	组成部分	主要职责
表现层	普通用户端、后台管理端	提供页面展示、表单输入、视频播放和用户交互入口
网关层	网关服务	统一入口、请求路由、基础过滤和内部接口隔离
业务服务层	主站、后台、资源、互动服务	完成用户、视频、文件、互动和管理业务处理
AI 能力层	AI 审核服务	消费审核任务，执行内容分析并返回审核结果
数据支撑层	MySQL、Redis、Elasticsearch、RabbitMQ、Seata、文件目录	保存业务数据、缓存、搜索索引、消息、事务状态和媒体资源

频最终发布结果，正式发布仍然由后台审核流程控制。

4.2 微服务模块划分

4.2.1 模块划分依据

删除4.2.1节

根据需求分析，系统主要包括用户访问、视频投稿、资源处理、互动行为、后台审核和 AI 审核等业务。不同业务的处理方式差异较大，因此需要按照职责进行服务拆分。领域驱动设计强调围绕业务领域识别边界，这为系统模块划分提供了参考^[36]。

本系统后端服务主要包括网关服务、主站业务服务、后台管理服务、资源文件服务和互动服务。公共组件模块和基础实体模块不作为独立运行服务，而是为各业务服务提供通用工具、常量、枚举、实体和查询对象。AI 审核服务以独立 Python 服务运行，用于处理视频内容风险分析。

4.2.2 系统服务模块说明

系统服务模块及其职责如表 4-2 所示。表中列出了各模块名称、服务端口、网关前缀和主要职责。通过该表可以看出，系统没有将所有功能都放到主站服务中，而是将文件处理、互动行为和后台管理进行了拆分。

主站服务是业务中心，但不是所有功能的承载点。资源服务独立出来后，视频上传、分片合并和转码处理可以集中管理；互动服务独立出来后，评论、弹幕和用户行为不会与视频主业务过度耦合；后台服务独立出来后，管理员接口和普通用户接口也更容易区分。

表 4-2 系统服务模块及职责

模块名称	端口	网关前缀	主要职责
streama-cloud-gateway	7071	—	API 统一入口、路由转发、内部接口隔离、管理端过滤
streama-cloud-web	7072	/web	用户、首页、视频信息、投稿、搜索和 AI 审核任务编排
streama-cloud-admin	7070	/admin	管理端登录、分类管理、视频审核、审核结果查询
streama-cloud-resource	7074	/file	图片上传、视频分片上传、视频转码和 HLS 资源输出
streama-cloud-interact	7073	/interact	评论、弹幕、点赞、收藏、投币等互动行为处理
streama-cloud-common	—	—	Redis、搜索、异常处理、公共组件和工具类
streama-cloud-base	—	—	常量、枚举、基础实体、查询对象和公共 DTO
ai-service	7075	—	消费审核任务，执行多模态审核并回传结果

4.2.3 服务协作关系

各服务之间围绕具体业务场景协作。用户提交投稿信息时，主站服务负责保存投稿主数据和分 P 文件数据，并将待转码文件加入队列；资源服务消费队列后完成视频转码，并回写转码结果；当全部分 P 转码完成后，主站服务创建 AI 审核任务并发送消息；AI 服务审核完成后回传结果；管理员最终在后台完成复核与发布操作。

表 4-3 系统主要服务调用关系

调用方	被调用方	调用方式	主要用途
网关服务	主站/后台/资源/互动服务	路由转发	将外部请求转发到对应服务
主站服务	后台服务	同步调用	获取分类树和分类列表
主站服务	互动服务	同步调用	查询用户互动状态,联动处理互动数据
资源服务	主站服务	内部接口	回写视频转码结果和文件信息
主站服务	AI 审核服务	消息队列	发送审核请求并接收审核结果
后台服务	主站服务	内部接口	查询待审核视频并提交人工审核结果

这种调用关系将同步请求和异步任务进行了区分。对查询分类、查询互动状态等轻量业务，可以采用同步调用；对视频转码、AI 审核等耗时任务，则使用队列异步处理，避免用户请求长时间阻塞。

4.3 网关与接口设计

删除4.3节

4.3.1 网关路由设计

网关是系统对外提供访问的统一入口。普通用户端和后台管理端的请求都先进入网关，再由网关根据请求路径转发到对应服务。这样前端不需要直接维护多个后端服务地址，后端服务也可以隐藏在网关之后。Spring Cloud Gateway 支持基于路径断言和过滤器进行路由控制，适合承担统一入口职责^[17]。

表 4-4 网关路由设计

请求前缀	目标服务	主要用途
/web/**	streama-cloud-web	用户端业务、视频信息、投稿、搜索等接口
/admin/**	streama-cloud-admin	管理端登录、分类管理、视频审核等接口
/file/**	streama-cloud-resource	图片上传、视频上传、视频资源访问等接口
/interact/**	streama-cloud-interact	评论、弹幕、点赞、收藏、投币等互动接口

由表 4-4 可以看出，系统通过路径前缀实现了对服务入口的基本划分。后台管理接口单独使用 /admin 前缀，有助于后续进行管理员过滤和权限控制；资源类接口单独使用 /file 前缀，也便于前端区分文件上传、封面访问和视频播放资源。

4.3.2 外部接口与内部接口划分

系统接口可以分为外部接口和内部接口。外部接口面向前端页面，例如用户登录、视频列表查询、分片上传和评论发送等；内部接口主要用于服务之间调用，例如资源服务转码完成后回写视频文件状态，后台服务调用主站服务执行审核操作。

为了避免内部接口被外部用户直接访问，系统对内部接口进行隔离。内部接口统一使用特定前缀，例如 /innerApi。网关在接收到外部请求时，如果发现请求路径包含内部接口前缀，则直接拒绝访问。该设计能够减少内部业务接口暴露带来的安全风险。

接口隔离的重点不是增加接口数量，而是明确哪些接口可以被前端访问，哪些接口只能在服务内部调用。该设计对后台审核、转码回写和内部数据联动等场景比较重要。

4.3.3 接口规范设计

接口命名应尽量保持清晰和一致。用户端接口、后台接口、资源接口和互动接口分别通过不同路径前缀区分，接口内部再按照业务动作组织。接口响应也应保持统一格式，无论是查询成功、操作失败还是权限不足，后端都返回统一结构，通常包括状态码、提示信息和业务数据。统一响应格式可以减少前端重复判断，也方便统一处理错误提示。

表 4-5 接口设计规则

设计项	设计说明
路径前缀	使用 <code>/web</code> 、 <code>/admin</code> 、 <code>/file</code> 、 <code>/interact</code> 区分服务入口
内部接口	使用 <code>/innerApi</code> 作为内部接口前缀，并由网关阻断外部访问
响应格式	返回统一状态码、提示信息和业务数据，减少前端重复处理
权限控制	普通用户接口和后台管理接口分开校验，避免权限混用
错误处理	对参数错误、登录失效、权限不足和业务异常返回明确提示

4.4 事务与一致性设计

删除4.4节

在线视频平台的业务链路中既存在单服务本地写操作，也存在跨服务协作写操作。例如，用户投稿、审核状态流转、互动计数变化和投币行为都可能同时涉及主站服务、资源服务、互动服务或后台管理服务。若某一环节调用失败，可能出现部分数据已提交而其他数据未更新的问题，因此系统需要在总体设计阶段区分强一致和最终一致两类场景。

对于关键跨服务写操作，系统引入 Seata AT 模式作为分布式事务支撑。Seata 通过事务协调器管理全局事务，并结合业务数据库中的 `undo_log` 记录完成异常回滚支持^[19]。在本系统中，Seata 不作为独立业务功能展示，而是作为跨服务写操作的数据一致性保障机制。

对于视频转码完成回写、播放计数累加、搜索索引同步、文件删除等不要求立即强一致的场景，系统采用异步队列、状态字段和补偿处理方式实现最终一致。这

样既避免所有业务都依赖强事务导致系统复杂度上升，也能保证核心写操作具有较好的可靠性。

4.5 数据库总体设计

修改为：4.5 数据库设计

4.5.1 数据库设计原则

删除4.5.1节内容

数据库设计是系统总体设计的重要组成部分。在线视频分享平台涉及的数据类型较多，包括用户信息、分类信息、视频信息、投稿信息、视频文件信息、评论弹幕、用户行为、AI 审核任务和审核明细等。如果数据表设计混乱，后续业务流程容易出现状态不一致和查询困难的问题。

本系统采用“单库逻辑分域”的设计方式，即核心业务表保存在同一个 MySQL 数据库中，但按照业务领域划分为不同数据域。这样既便于毕业设计阶段部署和演示，也能体现不同业务之间的数据边界。MySQL 适合保存结构化业务数据和具有明确关系的数据表^[20]。

数据库设计遵循以下原则：第一，核心业务数据优先保证结构清晰；第二，投稿态数据和正式发布数据分离，避免未审核视频直接进入前台；第三，AI 审核结果采用任务表和明细表保存，便于记录整体结论和分 P 级风险信息；第四，互动数据与视频主数据分离，减少高频行为对视频主表的影响。

4.5.2 核心数据表设计

1. 4.5.2和4.5.3合并为一个小节。
2. 在4.5.2小节中，删除表4-6，把该表中的每个数据表，画出其对应的实体属性图，并简述每章表格存储数据的结构、数据内容、数据作用等。
3. 之后，画出所有表格对应的ER图，并进行简要描述。

系统核心数据库表按业务数据、投稿数据、互动数据和正式数据划分。更适合说明各类数据表的业务职责，具体字段实现放在后续详细设计中说明。

由表 4-6 可以看出，系统数据表不是按页面简单划分，而是按业务状态和数据用途划分。视频有投稿态表和正式表，AI 审核有任务主表和任务明细表，互动行为也有独立数据表。这样的设计可以让视频生命周期更加清楚。

4.5.3 投稿数据与正式数据分离设计

视频数据采用“投稿表”和“正式表”分离的设计。用户提交或编辑视频后，数据先进入 video_info_post 和 video_info_file_post。其中，投稿主表保存标题、分类、标签、简介、状态等信息，投稿文件表保存每个分 P 的文件信息、上传标识、转码结果和播放资源路径。

表 4-6 核心数据库表及作用

业务域	数据表	主要作用
分类管理	category_info	保存视频一级/二级分类、分类编码、图标和排序
用户域	user_info、user_focus	保存用户基础信息、登录相关信息和关注关系
视频正式数据	video_info、video_info_file	保存审核通过后正式发布的视频主信息和分 P 文件信息
投稿数据	video_info_post、video_info_file_post	保存投稿态视频、转码状态、待审核状态和分 P 文件处理结果
互动数据	video_comment、video_danmu、user_action	保存评论、弹幕、点赞、收藏、投币等用户行为
AI 审核数据	ai_audit_task、ai_audit_task_item	保存 AI 审核任务、审核版本、风险等级、命中标签和命中片段

只有当视频完成转码并通过管理员审核后，系统才会将投稿表中的数据同步到正式视频表 video_info 和正式文件表 video_info_file。正式表中的数据才会被前台列表、搜索和播放功能使用。这样可以避免未审核视频或转码失败视频直接出现在用户端。

4.5.4 AI 审核数据设计

AI 审核数据采用主从结构。ai_audit_task 保存视频级审核任务,例如请求编号、视频编号、审核版本、任务状态、AI 建议、风险等级和审核摘要等信息;ai_audit_task_item 保存分 P 级审核结果,例如文件编号、风险分数、风险标签、命中片段和原因说明等信息。

采用主从结构的原因是,一个视频可能包含多个分 P 文件, AI 审核既需要给出整条视频的综合结论,也需要保留每个分 P 的风险细节。如果只保存一个总结果,管理员难以定位风险出现在哪个视频片段;如果只保存明细而没有总结果,审核列表展示又不够直观。

表 4-7 AI 审核数据设计

数据表	数据粒度	主要字段含义
ai_audit_task	视频级任务	请求编号、视频编号、审核版本、任务状态、AI 建议、风险等级、审核摘要
ai_audit_task_item	分 P 文件级结果	文件编号、风险分数、风险标签、命中片段、关键帧路径、原因说明

投稿数据与正式数据分离设计图

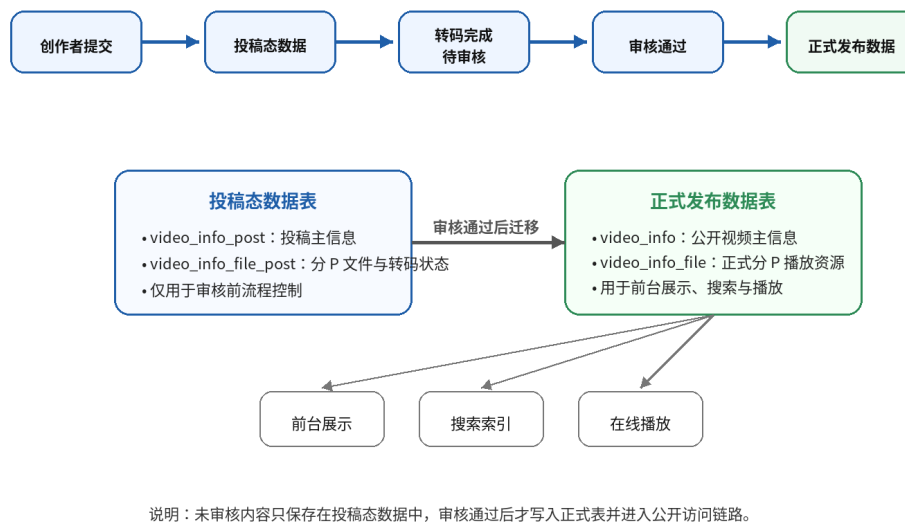


图 4-1 投稿数据与正式数据分离设计图

4.6 缓存、搜索与文件存储设计

删除 4.6 节的全部内容

4.6.1 Redis 缓存与临时状态设计

Redis 在系统中主要承担缓存、登录态、上传临时状态和轻量队列功能。相比数据库，Redis 更适合保存访问频繁、生命周期较短或需要快速读写的数据。Redis 支持多种数据结构，适合处理验证码、Token、上传状态和队列等场景^[11]。

原章节中对 Redis 键空间列举较细，容易与实现章节重复。修改后仅保留用途层面的设计说明，如表 4-8 所示。具体键名、过期时间和数据结构可以放在第五章实现部分展开。

4.6.2 视频搜索索引设计

视频搜索是在线视频平台的重要功能。用户可能根据标题、标签、简介或分类查找视频，如果只依赖 MySQL 模糊查询，搜索效率和相关性排序都不够理想。因此，系统采用 Elasticsearch 建立视频搜索索引。Elasticsearch 适合全文检索和相关性排序，可以为内容搜索提供更好的支持^[12]。

系统默认索引名称为 streama_video。视频审核通过或更新后，系统将视频标题、标签、简介、分类、播放数、弹幕数、收藏数等信息写入索引。用户搜索时，系

表 4-8 Redis 主要用途设计

用途类别	保存内容	设计目的
验证码与登录态	图形验证码、用户 Token、管理员 Token	减少数据库访问，支持登录校验和会话过期控制
分类缓存	首页分类、投稿分类、后台分类树	降低分类数据的重复查询开销
上传临时状态	上传标识、分片数量、临时路径、上传进度	支持大文件分片上传和断点处理
异步轻量队列	待转码任务、播放计数、待删除文件	支持后台任务逐步消费和延迟处理
热点统计	搜索关键词、播放量累加数据	支持搜索热度和访问统计的异步更新

统优先查询 Elasticsearch，并根据关键词匹配和排序条件返回结果。

表 4-9 视频搜索索引字段设计

字段类别	字段示例	设计用途
基础信息	视频编号、标题、封面、作者编号	用于搜索结果展示和视频跳转
分类信息	一级分类、二级分类、标签	用于分类筛选和标签匹配
文本信息	简介、标题、标签文本	用于关键词检索和相关性匹配
统计信息	播放量、弹幕数、收藏数、发布时间	用于排序和热度展示
状态信息	发布状态、审核状态	保证只检索正式发布内容

搜索索引保存的是面向检索和展示的冗余数据，而不是替代数据库中的正式业务数据。MySQL 保存权威数据，Elasticsearch 保存检索数据，两者通过视频发布和更新流程保持同步。

4.6.3 文件存储目录设计

视频分享平台除了保存数据库记录，还需要保存大量文件资源，包括封面图片、上传分片、转码后视频、HLS 切片和 AI 审核截图。文件存储设计如果不清晰，后续查找文件和清理临时文件都会比较困难。

本系统采用本地文件目录方式保存媒体资源。该方式实现简单，适合毕业设计本地运行和演示。根目录由项目配置指定，下级目录按文件用途进行划分。表 4-10 展示了主要文件目录设计。

表 4-10 文件存储目录设计

目录	保存内容	说明
cover/	视频封面和图片资源	用于前台视频卡片和详情页展示
temp/	上传分片临时文件	用于保存未合并的视频分片
video/	转码后视频资源	保存 HLS 播放列表和视频切片
audit-snapshot/	AI 审核截图	保存命中风险片段的关键帧截图

后续如果系统部署到多实例环境，可以将本地文件存储替换为对象存储，例如 MinIO 或云存储服务。当前阶段采用本地目录，更适合开发和演示。

4.7 消息通信与 AI 审核任务设计

4.7.1 异步任务总体设计

1. 4.7.1节和4.7.3节合并为4.7.1节，并且修改为：消息通信设计

视频平台中有些任务处理时间较长，不能让用户请求一直等待。视频转码需要读取文件、调用 FFmpeg 并生成切片；AI 审核需要提取音频、抽取关键帧并执行模型分析。这些任务都适合设计为异步流程。

本系统中的异步任务主要包括视频转码任务和 AI 审核任务。视频转码任务通过 Redis 队列进行处理，资源服务从队列中取出任务后执行转码；AI 审核任务通过 RabbitMQ 进行处理，主站服务发送审核请求，AI 服务消费消息并返回结果。消息队列能够让生产者和消费者解耦，适合处理跨服务异步任务^[13]。

由图 4-2 可以看出，异步任务设计的核心是让耗时操作脱离用户请求主流程。用户提交投稿后，系统先记录投稿数据，再由后台逐步完成转码和审核。这样既能减少用户等待，也能让任务失败时有机会记录状态并进行处理。

4.7.2 AI 审核消息通道设计

这一节内容要详细描述。
1. 用了哪些AI的算法。
2. 如何利用AI算法进行通道设计。
把上面2点内容，详细描述。

AI 审核任务采用 RabbitMQ 进行消息通信。生产者，将视频编号、分 P 文件信息、资源路径和任务编号等信息发送到审核请求队列；AI 服务作为消费者，从队列中获取任务并执行审核；审核完成后，AI 服务将结果发送到审核结果队列，主站服务再消费结果并写入数据库。

该消息通道设计包含请求投递、任务消费、结果回传和失败处理。即使 AI 服务暂时不可用，请求消息也可以保留在队列中，等待服务恢复后继续处理。

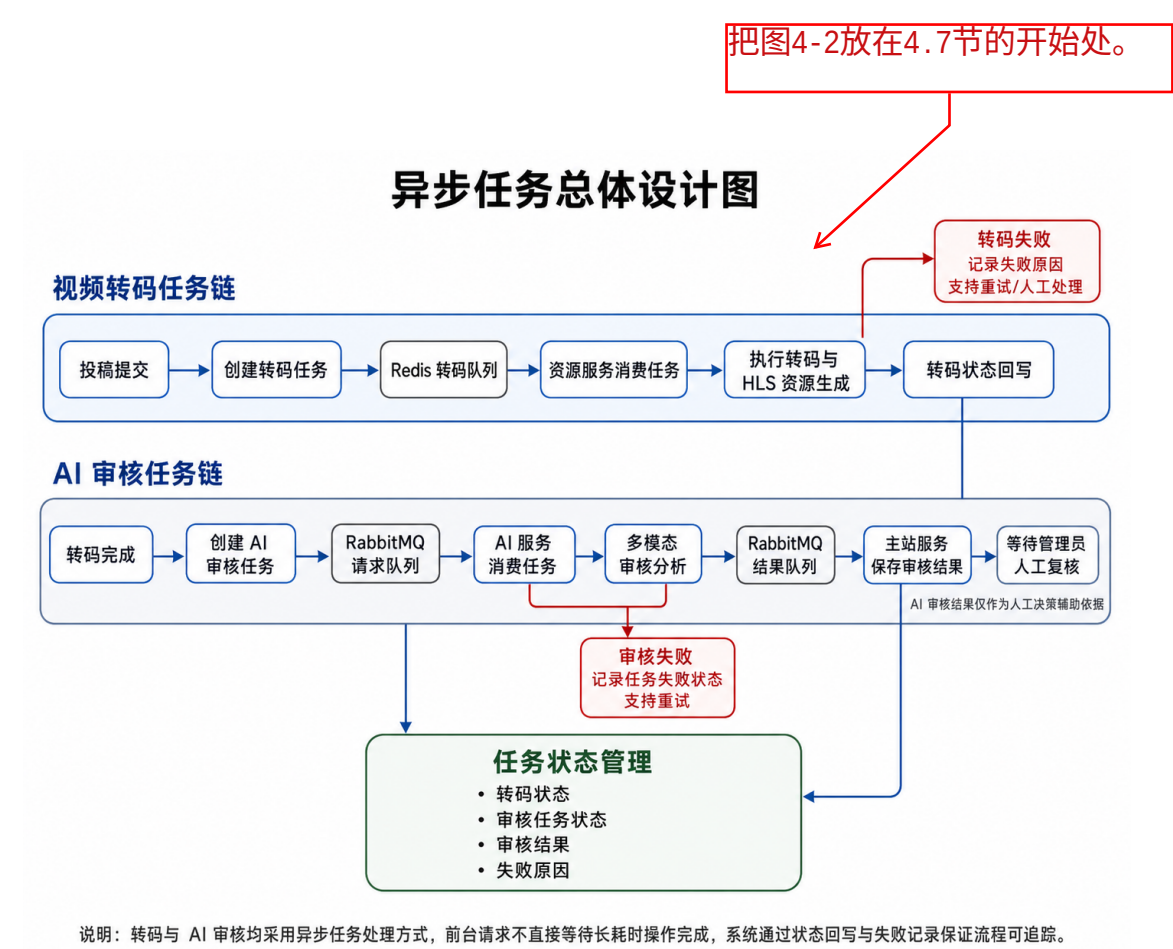


图 4-2 异步任务总体设计图

表 4-11 AI 审核消息通道设计

消息组件	名称	作用
Exchange	streama.audit.exchange	AI 审核请求和结果共用交换机
请求队列	streama.audit.request.queue	AI 服务消费审核请求
请求路由键	audit.video.request	主站服务发送审核请求时使用
结果队列	streama.audit.result.queue	主站服务消费审核结果
结果路由键	audit.video.result	AI 服务发送审核结果时使用
请求死信队列	streama.audit.request.dlq	保存请求处理失败的消息
结果死信队列	streama.audit.result.dlq	保存结果处理失败的消息

4.7.3 任务状态与异常处理思路

异步任务必须有明确状态。视频转码任务需要记录是否转码成功，AI 审核任务需要记录是否待处理、处理中、处理成功或处理失败。如果没有状态记录，系统很难判断任务是否完成，也无法在异常情况下进行恢复。

表 4-12 异步任务状态设计

状态	含义	处理说明
待处理	任务已创建但尚未开始	等待队列投递或等待消费者处理
处理中	任务已投递并正在处理	记录处理开始时间，避免重复处理
处理成功	任务完成并已返回结果	写入结果数据，更新业务状态
处理失败	任务执行或消息处理异常	记录失败原因，必要时转人工处理或重试

任务状态不是简单的辅助字段，而是长流程业务的控制依据。状态设计清楚后，系统才能在转码失败、审核失败或消息异常时进行定位和处理。

4.8 核心业务流程设计 ← 修改为：4.8 视频业务流程设计

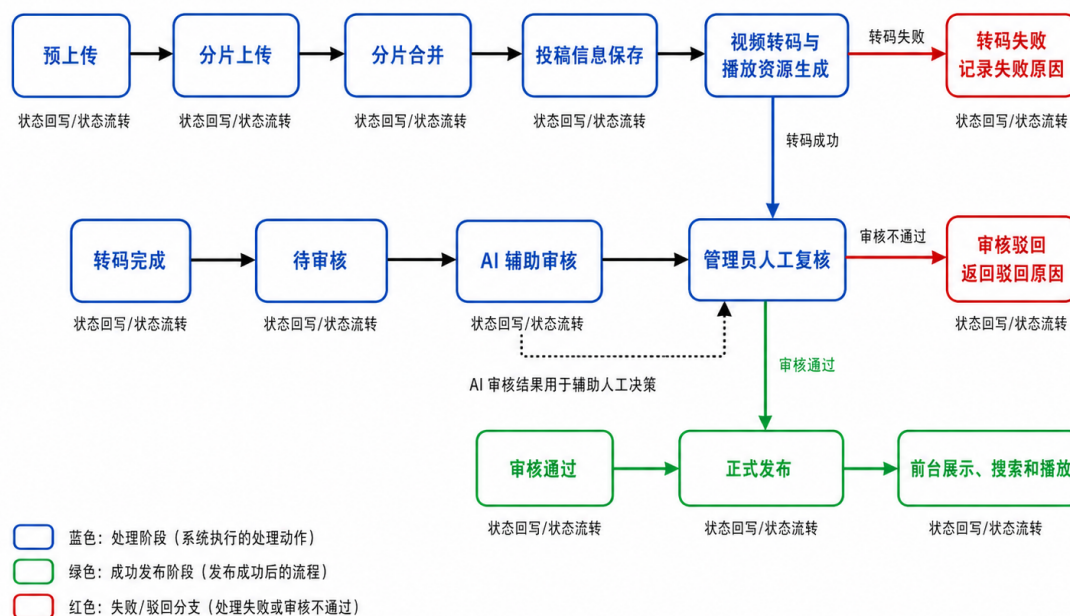
4.8.1 视频投稿、转码与审核流程

视频投稿流程从创作者选择视频开始。由于视频文件通常较大，系统采用预上传和分片上传方式。创作者选择文件后，先向资源服务申请上传任务，资源服务生成上传标识并记录上传状态；随后前端按分片上传视频文件，资源服务将分片保存到临时目录；所有分片上传完成后，主站服务保存投稿信息，并将待转码文件加入转码队列。

转码完成后，系统根据所有分 P 文件的转码状态决定后续流程：如果存在转码失败文件，则将投稿状态更新为转码失败；如果全部转码完成，则创建 AI 审核任务并等待审核结果。AI 审核完成后，系统将结果保存到数据库，由管理员在后台进行人工复核。管理员审核通过后，投稿态数据才会写入正式视频表和搜索索引。

图 4-3 仅从总体设计角度描述视频生命周期，不再展开每一步的代码处理逻辑。具体的分片上传、转码回写、AI 请求发送和审核发布实现将在第五章进行说明。

视频生命周期总体流程图



说明：视频在完成上传、转码、审核和发布后，才进入前台展示、搜索与播放阶段；
系统通过状态流转记录视频在各阶段的生命周期变化。

图 4-3 视频生命周期总体流程图

4.8.2 视频状态流转设计

视频状态是连接投稿、转码、审核和发布流程的重要依据。系统通过状态字段区分视频当前所处阶段，避免未完成转码或未通过审核的视频进入公开视频集合。

由表 4-13 可以看出，视频状态贯穿投稿、转码和审核过程。状态字段不仅用于展示给创作者查看，也是系统控制后续流程的重要依据。

4.8.3 发布与搜索同步设计

管理员审核通过后，系统需要将投稿态数据转换为正式发布数据。主站服务将 `video_info_post` 和 `video_info_file_post` 中的数据同步到 `video_info` 和 `video_info_file`。同步完成后，视频才可以在前台列表和播放页中展示。

同时，系统需要将视频信息写入 Elasticsearch 索引。搜索索引中保存标题、标签、简介、分类和统计字段，便于用户进行关键词搜索。正式数据和搜索索引之间需要保持基本一致，如果视频被删除或更新，也需要同步更新索引。

表 4-13 视频状态流转设计

状态	触发条件	后续处理
投稿待处理	创作者提交投稿信息	等待视频转码任务处理
转码中	资源服务开始处理视频文件	持续回写分 P 文件转码结果
转码失败	至少一个分 P 文件转码失败	记录失败原因，提示创作者处理
待审核	所有分 P 文件转码完成	创建 AI 审核任务并等待人工复核
审核通过	管理员确认视频可以发布	写入正式视频表和搜索索引
审核不通过	管理员驳回投稿	保留驳回状态和原因

表 4-14 视频发布与搜索同步设计

处理步骤	处理内容	设计目的
人工审核通过	管理员确认视频允许发布	保证内容经过人工确认
投稿数据迁移	将投稿表数据写入正式视频表	使视频进入公开视频集合
文件数据迁移	将投稿文件数据写入正式文件表	支持播放页读取分 P 信息
搜索索引写入	将标题、简介、标签和统计信息写入索引	支持用户关键词检索
状态更新	更新投稿状态和审核状态	保证创作者可以查看结果

审核通过并不是流程结束，还需要完成正式表写入和搜索索引同步。这样才能保证视频既能在前台列表和播放页中展示，也能被用户通过关键词搜索到。

4.9 安全与异常处理设计

删除 4.9 节的全部内容

4.9.1 权限控制设计

系统权限控制主要包括游客访问、普通用户认证、管理员认证和内部接口保护。游客可以访问公开视频列表、搜索和详情等基础内容；普通用户登录后可以进行评论、弹幕、点赞、收藏和投稿等操作；管理员登录后可以进入后台管理系统；内部接口只能由服务之间调用，不能被外部用户直接访问。

通过区分不同接口类型和访问对象，可以减少越权访问风险。尤其是后台审核、分类管理和 AI 审核结果查询等接口，不应与普通用户接口混用权限校验逻辑。

4.9.2 异常场景处理设计

系统运行过程中可能出现多种异常情况，例如视频分片上传失败、视频转码失败、AI 审核消息发送失败、审核结果回写失败、搜索索引写入失败等。如果没有异

表 4-15 权限控制设计

接口类型	访问对象	控制方式
游客接口	未登录用户	允许访问公开视频列表、搜索和详情等基础内容
用户接口	登录用户	校验普通用户 Token，允许互动和投稿操作
管理接口	管理员	校验管理员登录态，限制后台功能访问
内部接口	服务间调用	使用内部接口前缀，并由网关阻止外部直接访问

常处理，系统状态可能会停在中间阶段，用户和管理员都无法判断问题原因。

异常处理设计的重点是记录状态和错误原因。对于视频转码失败，系统应将投稿状态更新为转码失败，并保留失败提示；对于 AI 审核失败，系统可以记录任务失败状态，并允许管理员进行人工复核；对于消息处理失败，可以借助死信队列保存异常消息，后续再排查。

表 4-16 主要异常场景及处理方式

异常场景	可能影响	处理方式
分片上传失败	视频无法完整合并	保留上传状态，允许重新上传失败分片
视频转码失败	视频无法正常播放	更新转码失败状态，提示创作者处理
AI 审核请求发送失败	AI 初审无法执行	记录任务失败原因，支持重新发送或人工处理
AI 审核结果回写失败	后台无法查看审核结果	记录异常日志，结合消息重试或人工排查
搜索索引写入失败	视频发布后搜索不到	保留正式数据，后续补偿写入索引
内部接口被外部访问	可能造成越权调用	网关直接拦截并返回异常

异常处理不只是返回错误信息，更重要的是保证业务状态不混乱。只要状态和错误原因记录清楚，后续就可以通过重试、补偿或人工处理恢复业务流程。

4.10 本章小结

本章对在线视频分享平台的总体设计进行了说明。首先，根据系统需求确定了前后端分离与微服务结合的总体架构，并从表现层、网关层、业务服务层、AI 能力

层和数据支撑层分析了系统分层结构。随后，对系统后端服务模块进行了划分，明确了网关服务、主站服务、后台服务、资源服务、互动服务、公共模块、基础模块和 AI 审核服务的职责。

接着，本章对网关与接口设计进行了说明，包括路由规则、外部接口与内部接口边界、统一响应和权限控制等内容。在事务设计方面，本章区分了 Seata 支撑的跨服务强一致场景与异步队列支撑的最终一致场景。在数据库设计方面，本章采用单库逻辑分域方式，重点说明了核心表、投稿数据与正式数据分离、AI 审核主从表等设计。缓存、搜索和文件存储部分则分别说明了 Redis 用途、视频搜索索引和本地资源目录的设计思路。

最后，本章围绕视频生命周期设计了投稿上传、转码、AI 审核、人工复核、正式发布和搜索同步等核心业务流程，并对权限控制和异常处理进行了补充说明。通过本章设计，系统从需求分析阶段进入了可实现的结构设计阶段，也为第五章的详细设计与实现提供了基础。

第5章的内容存在很大问题。

1. 本章是给出详细设计，但5.1节~5.9节，仅是对功能进行描述，没有给出设计的具体内容。以5.7节AI审核为例。AI审核要给出详细设计中，采用了哪些AI算法用户审核。你的AI算法是什么？把审核的过程，用算法形式描述出来，并且画出流程图、活动图、序列图等。而不仅仅是简单的介绍AI审核功能。

2. 第5章的 5.1节~5.9节，都要根据上面的要求，重新写。

3. 第5章的 5.10节，删除。

删除文字。

第 5 章 系统详细设计与实现

在第四章中，本文已经完成系统总体架构、服务划分、数据库结构、缓存搜索、文件存储和消息通信等方面的设计。本章在总体设计基础上，进一步说明系统主要功能模块的详细实现过程。与总体设计章节不同，本章重点关注功能落地方式，主要包括用户端功能、创作中心投稿、视频分片上传、视频转码、互动行为处理、后台管理、AI 审核任务编排以及人工审核协同流程。

本系统采用前后端分离与微服务相结合的实现方式。用户端主要承担视频浏览、播放、投稿和互动功能；后台管理端主要承担分类维护、稿件审核和内容管理功能；后端服务按照业务职责拆分为网关服务、主站服务、资源服务、互动服务、后台管理服务 and 公共模块；AI 审核服务以独立服务形式运行，通过消息队列和 HTTP 接口与 Java 后端进行任务交互。通过服务拆分，系统能够避免业务逻辑集中在单一模块中，提高代码结构清晰度和后续维护能力。

5.1 用户端功能模块实现

5.1.1 用户端整体功能实现

用户端是普通用户访问视频分享平台的主要入口，主要包括首页浏览、视频播放、用户认证、创作中心和个人中心等功能。首页用于展示公开视频内容，视频播放页用于展示视频详情和播放资源，创作中心用于用户投稿，个人中心用于查看个人信息、作品状态和互动记录。

它的功能围绕用户访问路径展开。用户通常先进入首页浏览内容，再进入视频播放页观看视频；如果需要发表评论、弹幕、点赞、收藏或投币等操作，则需要登录；如果需要发布视频，则进入创作中心完成投稿。

5.1.2 首页浏览与搜索实现

首页模块主要负责展示平台中的公开视频内容。用户进入平台后，可以查看视频列表，并根据分类、关键词或排序条件筛选视频。首页展示的信息包括视频封面、标题、作者、播放量、弹幕数、发布时间等，这些信息能够帮助用户快速判断视频内容是否值得观看。

搜索功能主要面向关键词检索场景。视频平台中的搜索不能只依赖标题完全匹

配，因为用户可能根据标签、简介或部分关键词查找视频。因此，系统在视频审核通过后，将视频标题、简介、标签和统计字段写入搜索索引，搜索时再根据关键词匹配相关内容。首页浏览与搜索功能实现内容如表 5-1 所示。

表 5-1 首页浏览与搜索功能实现说明

功能点	实现内容	设计目的
视频列表展示	展示封面、标题、作者、播放量等信息	帮助用户快速浏览公开视频
分类筛选	根据一级或二级分类过滤视频	提高内容查找效率
关键词搜索	匹配标题、简介、标签等字段	支持用户主动查找内容
排序展示	按发布时间、播放量等字段排序	满足不同浏览习惯
热词记录	记录用户搜索关键词	为热词统计和后续推荐提供基础

首页模块承担内容入口作用。系统在实现时需要保证列表数据稳定加载，同时避免未审核或未发布的视频出现在首页和搜索结果中。因此，首页查询通常需要结合视频状态、发布时间、分类信息和统计字段进行过滤。

5.1.3 视频播放页实现

视频播放页是用户端的核心页面。播放页需要展示视频内容本身，也需要展示标题、作者、简介、标签、分类、播放量、点赞数、收藏数、投币数、评论数等信息。对于多分 P 视频，还需要提供分 P 切换入口，让用户可以选择不同视频片段播放。

由于上传视频经过资源服务转码后会生成 HLS 播放资源，播放页不直接播放原始视频文件，而是请求 .m3u8 播放列表，并由播放器按需加载媒体分片。这种方式可以降低完整大文件直接加载带来的压力，也更适合网页端流媒体播放场景。

播放页除了完成视频播放，还需要承载互动入口。用户可以在播放页进行点赞、收藏、投币、关注、发表评论和发送弹幕等操作。对于未登录用户，系统允许其观看公开视频，但涉及个人行为记录的操作需要提示登录。这样既保证公开视频的可访问性，也保证互动数据能够和用户身份对应。

5.1.4 用户认证与登录状态实现

用户认证模块主要用于识别用户身份，并为后续投稿、互动和个人中心功能提供权限基础。系统登录流程包括验证码校验、账号密码校验、登录令牌生成和登录

态缓存。用户登录成功后，后端生成身份令牌并写入 Redis，前端在后续请求中携带该令牌，网关或业务服务根据令牌解析用户身份。

为了避免将普通登录流程写得过于冗长，本文将认证实现要点整理为表 5-2。该模块虽然不是系统创新点，但它是保证投稿、互动和后台权限隔离的基础。

表 5-2 用户认证实现要点

实现环节	处理方式	作用
验证码校验	从缓存中读取验证码并比对	降低恶意登录和脚本请求风险
账号校验	校验账号、密码和用户状态	保证登录用户身份有效
令牌生成	登录成功后生成用户令牌	支持前后端分离场景下的身份传递
登录态缓存	将令牌与用户信息写入 Redis	提高身份校验效率
退出登录	删除缓存中的登录态	避免令牌继续使用

通过上述实现，系统能够在用户浏览公开视频时保持较低访问门槛，在用户进行投稿和互动时进行身份校验，从而兼顾开放访问和权限控制。

5.2 创作中心与视频投稿实现

5.2.1 创作中心功能结构

创作中心是用户发布视频内容的入口，主要包括视频投稿、稿件编辑、分 P 管理、封面上传、稿件状态查看等功能。与普通播放功能不同，创作中心涉及大文件上传、视频元信息保存、审核状态流转和转码任务触发，因此是系统实现中的重点模块。

创作中心的核心目标是使用户能够按照固定流程完成投稿，并能够查看稿件当前状态。视频从用户提交到最终发布，需要经过文件上传、投稿信息保存、转码处理、AI 审核和人工审核等环节。创作中心主要负责用户侧的投稿入口，具体的文件处理和审核任务则由资源服务、AI 审核服务和后台管理服务协同完成。

图 5-1 展示了视频投稿的整体处理链路。第 4 章已经从总体设计角度说明了视频生命周期，本节进一步说明创作中心中与投稿和文件上传相关的关键实现。

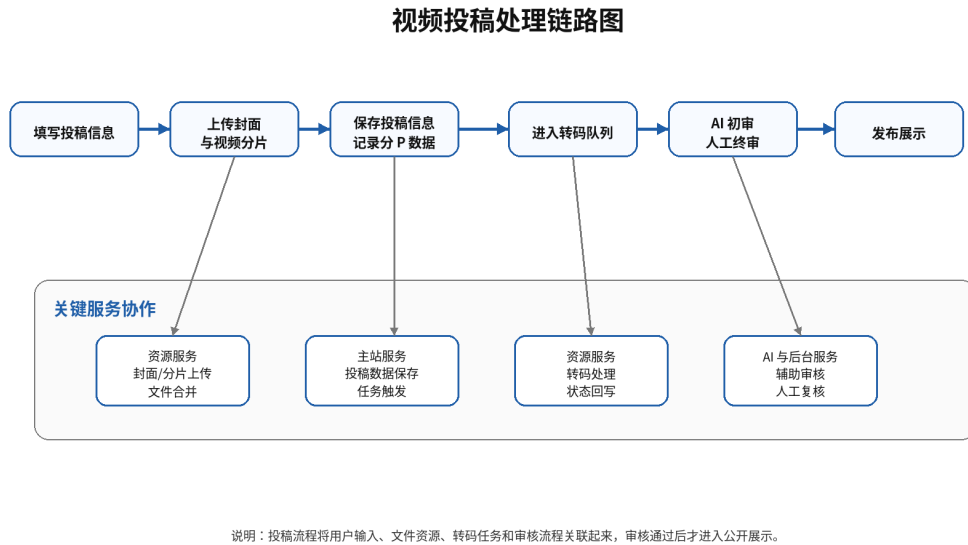


图 5-1 视频投稿处理链路图

5.2.2 视频分片上传实现

视频文件通常体积较大，如果直接一次性上传，容易受到网络波动、请求超时和服务器压力影响。为提高上传稳定性，系统采用分片上传方式。前端将视频文件切分为多个小片段，按顺序上传到资源服务；资源服务接收分片后暂存到指定目录，并记录分片上传状态；当所有分片上传完成后，再触发文件合并逻辑。

分片上传的关键在于保证每个分片能够被正确识别。系统需要记录文件唯一标识、分片序号、分片总数、上传用户和临时存储路径等信息。资源服务根据这些信息判断分片是否重复上传、是否全部上传完成，以及是否可以开始合并。视频分片上传处理流程如图 5-2 所示。

分片上传实现中需要重点处理三个问题：第一，重复分片应避免覆盖异常；第二，缺失分片应阻止合并；第三，合并完成后应及时清理临时文件，避免占用磁盘空间。系统通过分片状态记录和合并前校验保证上传流程可靠。

5.2.3 投稿信息保存实现

视频文件上传完成后，系统还需要保存投稿元信息。投稿信息包括视频标题、简介、分类、标签、封面、分 P 信息和作者信息等。为了便于审核和发布，系统将投稿状态与视频基础信息分离管理，审核通过前的视频不会直接进入公开列表。

视频分片上传处理流程图

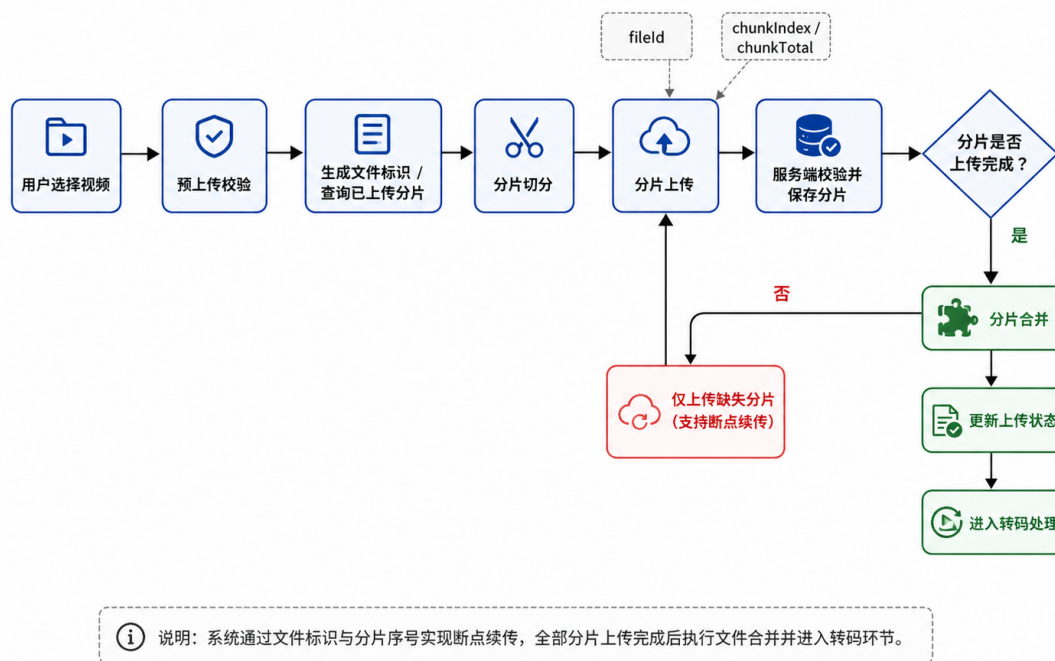


图 5-2 视频分片上传处理流程图

投稿信息保存时，后端需要完成参数校验、用户身份校验、分类有效性校验和分 P 信息校验。对于多分 P 视频，系统需要保证每个分 P 都关联到已上传的视频资源，避免出现只有元信息而没有播放资源的情况。投稿信息保存完成后，系统会根据业务规则触发后续转码和审核任务。

表 5-3 投稿信息保存实现内容

处理环节	实现内容	目的
参数校验	校验标题、分类、封面、分 P 信息是否完整	避免无效投稿进入流程
用户校验	根据登录态获取投稿用户身份	保证投稿归属明确
资源校验	校验上传视频与封面资源是否存在	避免投稿资源缺失
状态初始化	设置稿件初始审核和转码状态	支持后续状态流转
任务触发	写入转码或审核任务记录	推动后续异步处理

通过上述处理，投稿模块能够把用户输入、上传资源和审核任务关联起来，为

后续视频转码与内容审核提供完整数据基础。

5.3 视频转码与资源服务实现

5.3.1 资源服务职责

资源服务主要负责文件上传、文件存储、视频合并、视频转码和播放资源访问。由于视频资源体积较大，如果将文件处理逻辑放在主站服务中，会增加主站服务压力，也会影响普通业务接口响应。因此，系统将资源处理拆分为独立服务，使主站服务更加专注于业务数据管理。

资源服务接收到完整视频文件后，并不直接将原始文件暴露给用户播放，而是将其转码为适合网页播放的 HLS 格式。转码后的视频资源包括 .m3u8 播放列表文件和多个 .ts 媒体分片文件。前端播放器请求播放列表后，再按需加载媒体分片。

5.3.2 视频转码实现流程

视频转码是系统中的核心工程环节之一。系统在投稿文件合并完成后，将待转码任务写入异步队列，资源服务从队列中取出任务并执行转码操作。转码完成后，资源服务更新视频资源状态，并将播放地址写入对应记录中。视频转码实现流程如图 5-3 所示。

转码过程需要重点处理任务状态。任务开始时应标记为处理中，转码成功后标记为成功并记录播放资源地址；如果转码失败，则需要记录异常原因并更新失败状态，便于后台管理端查看和后续重试。这样可以避免用户提交的视频在异常情况下长期停留在不明确状态。

5.3.3 播放资源访问实现

视频转码完成后，播放页通过资源地址访问 HLS 播放列表。资源访问过程中需要保证路径有效性和文件可读性。对于公开发布的视频，系统允许用户端访问其播放资源；对于未审核、已驳回或已删除的视频，系统不应在前台返回播放地址。

资源服务在实现上需要处理文件路径映射问题。服务器内部保存的是实际磁盘路径，而前端需要访问的是可公开访问的资源 URL。因此，系统需要将内部文件路径转换为前端可访问地址，并保证地址与视频记录正确关联。

视频转码实现流程图

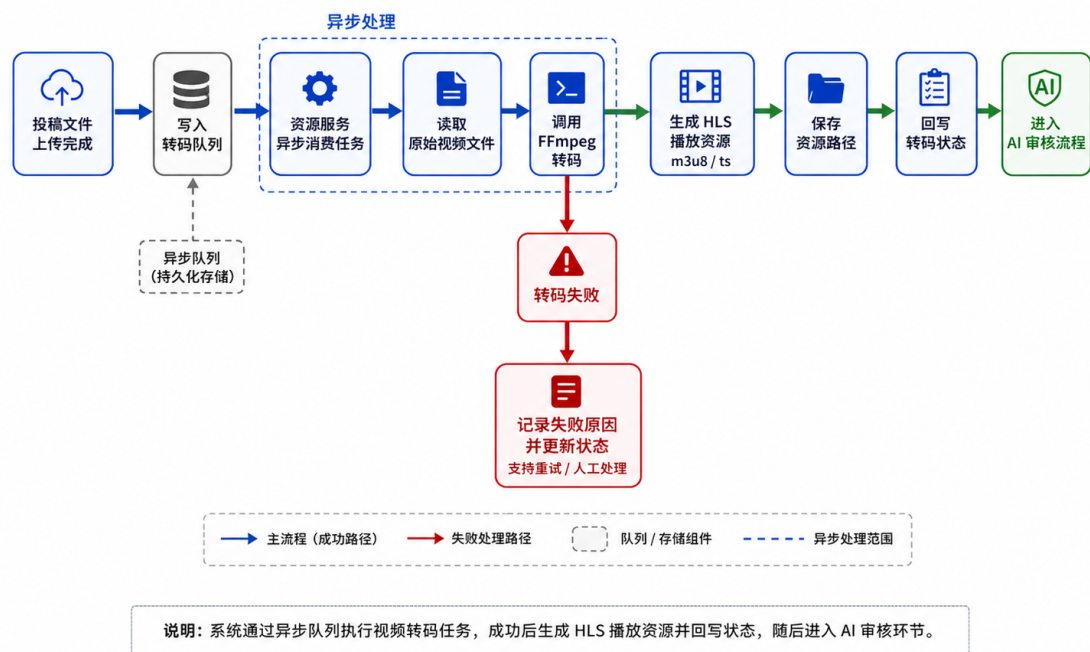


图 5-3 视频转码实现流程图

5.4 互动功能模块实现

互动功能用于增强用户参与感，主要包括点赞、收藏、投币、关注、评论和弹幕等。与视频上传和 AI 审核相比，互动功能的业务逻辑相对常规，因此本文不再使用冗长算法描述，而是从实现策略角度进行说明。

互动行为一般需要满足三个要求：第一，用户必须登录后才能执行个人行为；第二，同一用户对同一对象的互动应有明确规则，例如点赞可以取消，投币通常不能无限重复；第三，互动结果需要及时反馈到视频统计字段中，使播放页展示的数据保持一致。

互动功能虽然不是本文的技术重点，但它直接影响用户体验。系统在实现时通过互动服务集中处理相关接口，避免互动逻辑分散在主站服务中。对于播放量、点赞数、收藏数等统计字段，可以根据业务需要采用同步更新或异步汇总方式，从而在准确性和性能之间取得平衡。

表 5-4 互动功能实现策略

互动类型	实现策略	注意事项
点赞	记录用户与视频的点赞关系，支持再次点击取消	避免重复计数
收藏	记录用户收藏夹与视频关系	支持用户查看收藏列表
投币	记录用户投币行为并更新视频统计	需要限制重复投币规则
关注	记录用户与作者之间的关注关系	避免自己关注自己
评论	保存评论内容、用户、视频和时间信息	需要进行内容合法性校验
弹幕	保存弹幕内容、出现时间和视频分 P 信息	播放时按时间轴加载

5.5 跨服务事务与一致性实现

系统中部分业务并不局限于单个服务内部完成。例如，用户投币时需要记录互动行为，同时还需要更新用户资产和视频统计数据；视频审核通过时，需要将投稿表数据迁移到正式表，并同步更新搜索索引或相关状态。此类操作若仅依赖单个服务本地事务，无法覆盖远程服务调用失败带来的数据不一致问题。

为降低跨服务写操作中的一致性风险，系统在公共基础模块中引入 Seata，并在关键业务方法上结合全局事务机制控制事务边界^[19]。当跨服务调用链路中的某个环节出现异常时，全局事务能够协调相关分支事务回滚，从而减少投稿、审核、互动和投币等业务场景中的部分提交问题。

需要说明的是，并非所有操作都使用强事务。播放量累加、搜索索引更新、文件延迟删除等场景对实时一致性要求较低，系统仍采用异步队列和状态补偿方式处理。这样可以在一致性、性能和实现复杂度之间取得平衡。

5.6 后台管理功能实现

5.6.1 后台管理端整体实现

后台管理端主要面向系统管理员，用于完成分类维护、视频审核、视频管理和用户管理等操作。与普通用户端相比，后台管理端对权限控制要求更高，所有后台接口都需要校验管理员身份，避免普通用户访问内部管理功能。

后台管理功能通过独立的后台管理服务实现，并由网关根据路由规则转发请求。后台管理服务不直接处理视频文件，而是通过业务数据和资源地址查看稿件信息，通过审核接口控制视频状态流转。

5.6.2 视频审核管理实现

视频审核是后台管理端的核心功能。系统在用户投稿后生成待审核记录，管理员可以在后台查看稿件列表、稿件详情、AI 审核结果、视频基本信息和分 P 资源。管理员根据内容情况选择通过或驳回，并填写必要的审核意见。

后台审核的关键是状态控制。系统需要保证只有处于待审核或待人工审核状态的稿件才能被处理；已经发布、已驳回或已删除的稿件不能被重复审核。审核通过后，系统将视频状态更新为可发布状态，并同步搜索索引；审核驳回后，系统记录驳回原因，并在创作中心向投稿用户展示。

表 5-5 后台视频审核实现内容

功能环节	实现内容	作用
审核列表	按状态、分类、时间等条件查询投稿记录	方便管理员筛选稿件
审核详情	展示视频信息、分 P 资源和 AI 审核结果	支持人工判断
审核通过	更新视频状态并写入公开展示数据	使视频可被搜索和播放
审核驳回	记录驳回原因并更新稿件状态	向投稿用户反馈问题
权限校验	校验管理员登录态和接口访问权限	防止越权操作

通过后台审核管理，系统能够将自动化审核结果和人工判断结合起来，避免完全依赖 AI 模型导致误判，也避免人工从零开始检查所有内容造成效率低下。

5.6.3 分类与内容管理实现

分类管理用于维护平台视频分类结构。管理员可以新增、修改或删除分类信息，用户投稿时需要选择有效分类，首页浏览和搜索筛选也依赖分类数据。为了提高访问效率，分类信息可以在缓存中保存，当后台修改分类后再更新缓存。

内容管理主要用于查看已经发布的视频、处理异常稿件和维护平台内容秩序。对于已发布视频，管理员可以根据平台规则进行下架或删除处理。下架操作通常只改变视频展示状态，不一定立即删除资源文件；删除操作则需要配合资源清理策略，避免产生无效文件引用。

5.7 AI 审核任务编排实现

5.7.1 AI 审核任务创建

AI 审核是本系统的特色模块之一，用于在视频进入人工审核前对内容进行自动初审。AI 审核并不替代管理员最终判断，而是通过文本、音频和画面维度的风险识别，为人工审核提供参考依据。

用户投稿完成并生成视频资源后，系统会创建 AI 审核任务。任务创建时需要记录视频编号、分 P 编号、任务版本、审核状态、任务创建时间等信息。对于多分 P 视频，系统需要为每个分 P 建立审核明细，避免只审核其中一个片段而遗漏其他内容。

AI 审核任务创建需要避免重复提交。系统可以根据视频编号、分 P 编号和审核版本判断是否已经存在有效任务。如果任务已经创建，则不应重复写入队列，否则可能造成同一视频被多次审核，影响审核状态准确性。

5.7.2 消息队列与审核服务调用

AI 审核任务属于耗时任务，不适合在用户投稿接口中同步完成。如果投稿接口等待 AI 模型处理完成再返回，会导致用户等待时间过长，也容易受到模型服务状态影响。因此，系统使用消息队列解耦投稿流程和 AI 审核流程。

Java 后端创建审核任务后，将任务消息发送到 RabbitMQ。AI 审核服务监听对应队列，接收到任务后根据视频资源地址执行审核，并将审核结果回写到后端接口或数据库中。通过消息队列，投稿接口只需要完成任务创建和消息发送，不需要直接等待 AI 审核完成。

表 5-6 AI 审核任务消息字段说明

字段	含义	用途
任务编号	AI 审核任务的唯一标识	用于结果回写和状态更新
视频编号	被审核视频的唯一标识	关联投稿记录
分 P 编号	被审核片段的唯一标识	支持多分 P 审核
资源地址	视频或转码后资源的访问路径	供 AI 服务读取内容
审核版本	当前审核任务版本号	避免旧任务覆盖新结果
回调地址	Java 后端接收结果的接口地址	支持结果回写

消息队列的使用降低了 Java 后端与 AI 服务之间的耦合度。当 AI 服务暂时不可

用时，任务可以保留在队列或失败记录中，后续再进行重试或人工处理，从而提高系统整体稳定性。

5.8 多模态 AI 审核服务实现

5.8.1 AI 审核服务结构

AI 审核服务采用独立服务形式实现，主要负责接收审核任务、解析视频资源、执行多模态检测、生成风险结果并回写审核结论。该服务由 Python 实现，使用 FastAPI 构建 HTTP 健康检查接口，并通过 Uvicorn 启动运行；同时，服务会注册到 Nacos，并通过 RabbitMQ/Pika 消费审核请求、发送审核结果^[10,13,15,37-38]。由于 AI 模型推理和视频处理对运行环境要求较高，将其独立部署可以避免影响 Java 业务服务。

本系统中的 AI 审核主要从文本、音频和图像三个维度识别风险内容。文本维度可用于识别标题、简介、标签、字幕或语音转写文本中的敏感内容；音频维度可结合 Whisper、Silero VAD 和 YAMNet 等模型分析语音片段与声音事件；图像维度则可结合 Qwen-VL 对关键帧画面进行理解^[4,6,14,39]。多模态结果最终汇总为整体风险等级和审核建议。

图 5-4 展示了 AI 审核服务结构。该服务不直接决定视频最终能否发布，而是为后台管理员提供风险等级、风险类型和审核建议。

5.8.2 文本、音频与画面审核实现

文本审核主要处理视频标题、简介、标签以及语音识别结果。系统可以先对文本进行规范化处理，再判断是否包含风险表达。对于语音内容，AI 服务可以先将音频转写为文本，再复用文本审核逻辑，从而发现视频声音中的不合规内容。

音频审核主要用于识别异常声音或特定风险声音。由于音频模型推理可能受到背景音、噪声和音乐影响，系统不会仅依赖音频结果直接驳回视频，而是将音频风险作为综合审核依据之一。

画面审核通过视频抽帧实现。AI 服务按照一定时间间隔从视频中截取关键帧，并对画面内容进行识别。抽帧数量需要在审核准确率和处理耗时之间进行平衡。如果抽帧过少，可能遗漏风险画面；如果抽帧过多，则会增加审核耗时和计算资源占用。

通过表 5-7 可以看出，多模态审核并不是简单地输出单一判断，而是从多个角度

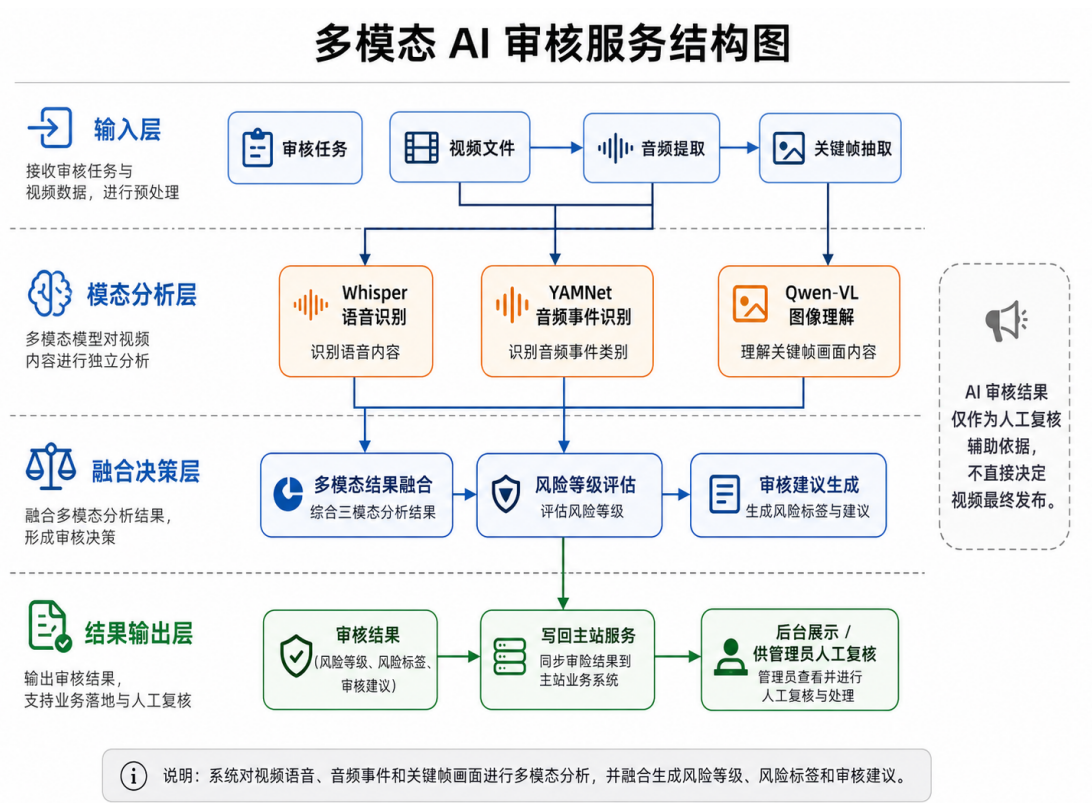


图 5-4 多模态 AI 审核服务架构图

表 5-7 多模态审核实现方式

审核维度	实现方式	输出结果
文本审核	检测标题、简介、标签和转写文本	文本风险类型与风险等级
音频审核	分析视频音频片段中的声音特征	音频风险提示
画面审核	对视频进行抽帧并识别画面内容	画面风险标签和证据帧
结果汇总	综合多个维度结果生成建议	通过、疑似风险或建议人工复核

形成审核参考。后台管理员可以结合 AI 审核结果和视频实际内容做最终决定。

5.8.3 审核结果回写实现

AI 审核完成后，需要将结果回写到 Java 后端。回写内容主要包括任务编号、分 P 编号、文本风险结果、音频风险结果、画面风险结果、综合风险等级、审核建议和证据信息等。Java 后端接收到结果后，更新审核任务状态，并在后台审核详情页展示相关信息。

审核结果回写需要注意版本一致性。如果用户重新编辑稿件或重新提交审核，旧版本 AI 任务可能仍然返回结果。为了避免旧结果覆盖新任务，系统需要在回写时校验任务版本。只有当前有效版本的审核结果才能更新稿件状态。

5.9 AI 初审与人工终审协同实现

5.9.1 人机协同审核机制

本系统采用“AI 初审 + 人工终审”的协同审核机制。AI 初审负责自动识别明显风险内容，并将风险信息展示给管理员；人工终审负责结合平台规则、上下文和实际内容作最终判断。该机制既提高了审核效率，又避免系统完全依赖模型判断。

人机协同审核的核心是职责划分。AI 审核更适合处理重复性、规则性和批量化识别任务，人工审核更适合处理语义复杂、上下文相关或边界模糊的内容。二者职责划分如表 5-8 所示。

表 5-8 AI 审核与人工审核职责划分

审核方式	主要职责	适用场景
AI 初审	自动分析文本、音频和画面风险	批量任务、初步筛查、风险提示
人工终审	根据平台规则做最终通过或驳回决定	边界内容、复杂语义、争议内容
协同机制	AI 提供证据，人工确认结论	提高效率并降低误判风险

通过人机协同，管理员不需要完全从零开始检查每个视频，而是可以优先关注 AI 标记出的风险片段。对于 AI 判断为低风险的视频，管理员仍可进行抽查或快速确认；对于 AI 标记为高风险的视频，管理员可以结合证据帧和风险说明进行重点审核。

5.9.2 审核状态流转实现

视频审核涉及多个状态，包括待转码、转码中、待 AI 审核、AI 审核中、待人工审核、审核通过、审核驳回等。系统需要通过状态字段保证不同处理环节有序执行。只有当视频资源准备完成后，AI 审核才有完整输入；只有当 AI 审核结果返回或进入人工复核状态后，管理员才适合进行最终审核。

状态流转实现中需要避免越级操作。例如，转码失败的视频不能进入发布状态；已经驳回的视频不能被普通用户直接访问；审核通过后才允许写入搜索索引并在前台展示。通过状态约束，系统能够保证视频生命周期清晰可控。

5.10 异常处理与安全控制实现

5.10.1 异常处理实现

系统运行过程中可能出现多种异常，包括上传分片缺失、文件合并失败、转码失败、AI 服务不可用、审核结果回写失败、用户越权访问等。为了提高系统稳定性，后端需要对不同异常进行分类处理，并向前端返回明确错误信息。

对于普通业务异常，系统可以通过统一异常处理器返回标准响应；对于文件处理和 AI 审核等异步任务异常，系统需要记录失败状态和错误原因，避免任务一直处于处理中。异常处理策略如表 5-9 所示。

表 5-9 系统异常处理策略

异常类型	处理方式	目的
参数异常	返回参数错误提示	阻止无效请求继续执行
权限异常	拦截接口并返回未授权提示	防止越权访问
上传异常	记录缺失分片或上传失败原因	支持重新上传或排查问题
转码异常	更新转码失败状态并记录日志	避免任务长期卡住
AI 审核异常	标记审核失败或转人工处理	保证审核流程可继续推进

通过异常分类处理，系统能够在出现问题时保持状态明确。管理员可以根据错误记录定位问题，用户也可以根据提示重新操作。

5.10.2 权限与接口安全实现

系统包含普通用户接口、后台管理接口和内部服务接口，不同类型接口的访问权限不同。普通用户接口主要面向登录用户和游客，后台管理接口只允许管理员访问，内部服务接口则应避免被外部直接调用。

系统通过登录态校验、角色判断、网关路由控制和服务内部校验共同实现权限控制。对于需要登录的接口，后端会解析请求中的令牌；对于后台接口，系统还需要确认用户是否具备管理员身份；对于 AI 回写或服务间调用接口，可以通过内部标识或访问白名单降低被非法调用的风险。

5.11 本章小结

本章在系统总体设计基础上，对视频分享平台的主要功能实现进行了说明。首先介绍了用户端的首页浏览、搜索、播放和用户认证实现；随后重点分析了创作中心中的视频投稿、分片上传和投稿信息保存；接着说明了资源服务中的视频合并、转码和播放资源访问；然后对互动功能、后台管理功能、AI 审核任务编排、多模态审核服务和人机协同审核机制进行了阐述；最后总结了异常处理和权限控制实现。

经过整理后，本章重点保留了能够体现系统工程复杂度和项目特色的部分，即视频分片上传、视频转码、AI 审核任务编排和人工审核协同机制。对于登录、互动等常规业务功能，则采用表格和简要说明方式呈现，避免与前文总体设计章节重复，也使章节内容更加聚焦于“系统如何实现”。

第 6 章 系统测试与结果分析

删除文字

系统测试的主要目的在于验证在线视频分享平台在实际运行环境下是否能够满足需求分析中提出的功能要求，并检查用户端、创作中心、后台管理端、资源文件服务、互动服务以及 AI 审核服务之间的协作是否稳定。本系统不是单一页面应用，而是由多个微服务、中间件和外部工具共同组成，因此测试时既要关注单个功能点是否正确，也要关注视频从投稿到发布的完整链路是否能够顺利流转。

本章围绕测试环境、测试方案、核心功能测试、视频处理测试、AI 审核测试、异常场景测试以及测试结果分析展开。为了避免与前文的需求分析、总体设计和详细实现章节重复，本章不再展开说明各模块的内部实现原理，而是重点呈现测试场景、测试步骤、预期结果、实际表现和测试结论。

6.1 测试环境与测试方案

6.1.1 测试环境配置

系统测试环境包括前端运行环境、Java 后端运行环境、Python AI 服务运行环境，以及数据库、中间件、分布式事务组件和视频处理工具等基础组件。测试前需要保证 MySQL、Redis、Elasticsearch、RabbitMQ、Nacos、Seata、FFmpeg 等组件能够正常启动，并依次启动网关服务、主站服务、后台管理服务、资源服务、互动服务和 AI 审核服务。

表 6-1 给出了本系统测试环境的主要配置。

由表 6-1 可以看出，系统运行依赖多个基础组件。若 RabbitMQ 未启动，则 AI 审核任务无法正常投递；若 Seata 未正常连接，则关键跨服务写操作可能无法完成全局事务协调；若 FFmpeg 路径配置错误，则视频转码流程无法完成；若 Elasticsearch 异常，则搜索功能无法返回正常结果。因此，在正式测试前需要先完成基础环境连通性检查。

6.1.2 测试方案设计

本系统采用功能测试、流程测试和异常测试相结合的方式进行验证。功能测试用于检查单个模块是否满足需求，例如登录、视频播放、评论、弹幕、分类管理等；流程测试用于检查跨模块业务链路是否顺畅，例如视频投稿后能否完成分片上传、转

表 6-1 系统测试环境配置

环境类别	配置项	说明
操作系统	Windows / Linux	用于本地开发、联调和功能演示
后端环境	JDK 17、Maven	运行 Spring Boot 与 Spring Cloud 微服务
前端环境	Node.js、Vue	运行用户端和后台管理端页面
AI 服务环境	Python、FastAPI	提供音频、图像和视频帧审核接口
数据库	MySQL 8	保存用户、视频、投稿、互动和审核数据
缓存组件	Redis	保存登录态、验证码、上传状态和部分队列数据
搜索引擎	Elasticsearch	支持视频标题、简介和标签检索
消息队列	RabbitMQ	传递 AI 审核任务和审核结果
注册中心	Nacos	管理微服务注册发现和配置
视频工具	FFmpeg	完成视频转码、切片和音频提取

码、AI 审核、人工审核和前台发布；异常测试用于检查系统在未登录访问、权限不足、文件格式错误、服务不可用等情况下是否能够给出合理处理。

表 6-2 展示了系统测试方案。

表 6-2 系统测试方案

测试类型	测试内容	测试目标
用户端功能测试	首页浏览、视频搜索、视频播放、评论、弹幕、点赞、收藏、投币	验证普通用户功能可正常使用
创作中心测试	封面上传、视频分片上传、投稿提交、稿件状态查询	验证创作者投稿流程完整
后台管理测试	管理员登录、用户管理、分类管理、视频审核、AI 审核结果查看	验证后台管理功能可用
视频处理测试	分片保存、文件合并、视频转码、HLS 文件生成、播放统计	验证资源处理链路正确
AI 审核测试	审核任务创建、消息发送、AI 服务处理、结果回写、后台展示	验证 AI 审核闭环可用
异常场景测试	未登录操作、权限不足、重复提交、转码失败、AI 服务异常	验证系统异常处理能力

本章测试重点放在核心业务链路上。对于登录注册、分类管理等常规功能，主要采用接口返回结果和页面显示结果进行验证；对于视频上传、转码、审核和发布等关键流程，则结合数据库状态、文件生成情况、消息队列消费情况和页面展示结

果进行综合判断。

6.2 核心功能测试

6.2.1 用户端功能测试

用户端功能主要面向普通用户和内容浏览者，测试范围包括首页视频列表、分类筛选、关键词搜索、视频播放、评论弹幕以及互动行为等。测试时分别使用未登录用户和已登录用户进行操作，以验证系统是否能够正确区分游客权限和登录用户权限。

表 6-3 给出了用户端主要功能测试结果。

表 6-3 用户端功能测试结果

测试项	测试操作	实际结果	结论
首页浏览	访问首页并切换分类	视频列表正常加载，分类切换后数据随之变化	通过
视频搜索	输入关键词进行检索	返回与标题、简介或标签相关的视频结果	通过
视频播放	打开视频详情页并播放视频	播放器能够加载视频资源并正常播放	通过
评论功能	登录后发布评论	评论内容保存成功并在页面中展示	通过
弹幕功能	登录后发送弹幕	弹幕内容按时间点显示在播放器区域	通过
互动功能	执行点赞、收藏和投币操作	互动状态和计数能够正确更新	通过
游客限制	未登录状态下执行互动操作	系统提示用户登录后再操作	通过

测试结果表明，用户端基础浏览和互动功能能够满足系统需求。未登录用户可以进行视频浏览和搜索，但在评论、弹幕、点赞、收藏和投币等需要身份识别的操作上会被系统拦截，符合权限控制要求。

6.2.2 创作中心功能测试

创作中心是系统中较为重要的功能模块，主要负责视频投稿、封面上传、视频分片上传和稿件状态查看。由于视频文件通常较大，测试时重点验证分片上传、文件合并和投稿状态流转是否正确。

第6.2、6.3、6.4节的内容需要重写。

软件测试主要内容是测试用例设计和测试数据。根据以下要求进行修改：

1. 系统测试的测试用例需要以表格形式给出，表格的结构包括以下各列：测试内容、测试数据、预期结果、实际结果，最后对测试结果进行总结评价。
2. 测试数据要根据测试功能，给出具体的数据。

表 6-4 展示了创作中心功能测试结果。

表 6-4 创作中心功能测试结果

测试项	测试操作	实际结果	结论
封面上传	上传符合格式要求的封面图片	图片上传成功, 页面可正常预览	通过
预上传	提交视频文件基本信息	系统返回上传任务标识和分片信息	通过
分片上传	按顺序上传视频分片	分片被保存至临时目录, 上传进度正常更新	通过
文件合并	所有分片上传完成后触发合并	系统生成完整视频文件并进入后续处理流程	通过
投稿提交	填写标题、分类、标签、简介并提交	投稿记录创建成功, 稿件进入待处理状态	通过
状态查询	在创作中心查看稿件状态	能够显示转码中、待审核、已发布或驳回等状态	通过

测试结果表明, 创作中心能够支持视频文件上传和投稿信息提交。系统可以根据上传进度记录分片状态, 在分片完整后完成文件合并, 并将投稿记录推进到后续转码和审核流程。

6.2.3 后台管理功能测试

后台管理端主要面向管理员, 测试内容包括管理员登录、分类维护、用户管理、视频审核、AI 审核结果查看和审核操作记录等。该部分测试重点是验证后台接口权限是否正确, 以及管理员能否根据视频内容和 AI 审核结果进行人工审核。

表 6-5 给出了后台管理功能测试结果。

测试结果表明, 后台管理端能够完成管理员日常管理和审核操作。普通用户无法越权访问后台接口, 管理员可以结合 AI 审核结果对视频进行人工终审, 满足系统对内容管理的需求。

6.3 视频处理与业务链路测试

6.3.1 视频投稿到发布链路测试

视频投稿到发布是系统最核心的业务链路, 涉及创作中心、资源文件服务、转码任务、AI 审核服务、后台管理服务和用户端展示等多个模块。为了验证链路完整

表 6-5 后台管理功能测试结果

测试项	测试操作	实际结果	结论
管理员登录	使用管理员账号登录后台	登录成功后进入后台管理页面	通过
分类管理	新增、修改和删除视频分类	分类数据正确保存, 用户端分类同步更新	通过
用户管理	查询用户列表并修改用户状态	用户状态变更后相关操作受到限制	通过
审核列表	打开待审核视频列表	能够查看待审核稿件和 AI 审核摘要	通过
审核详情	查看视频详情、封面和审核结果	页面正常展示视频信息与风险提示	通过
人工审核	执行通过或驳回操作	视频状态正确更新, 审核记录正常保存	通过
权限控制	普通用户访问后台接口	请求被拦截或返回无权限提示	通过

性, 测试时按照“用户投稿、系统处理、AI 初审、人工终审、前台展示”的顺序进行验证。

图 6-1 展示了视频投稿到发布的测试链路。

测试过程中, 系统能够在投稿完成后创建视频记录和转码任务; 转码完成后生成可播放资源; AI 审核服务能够接收任务并返回审核结果; 管理员审核通过后, 视频能够在用户端被搜索、浏览和播放。该测试说明系统核心链路可以形成闭环。

6.3.2 视频转码与播放资源测试

视频转码测试主要验证 FFmpeg 是否能够正确处理上传后的视频文件, 并生成 HLS 播放所需的资源文件。测试时选择不同大小和格式的视频进行上传, 检查转码任务执行状态、输出文件目录、播放地址和页面播放效果。

表 6-6 展示了视频转码与播放资源测试结果。

测试结果表明, 系统能够完成从原始视频文件到 HLS 播放资源的转换。对于异常视频文件, 系统能够记录转码失败状态, 从而避免用户端展示无法播放的视频。

6.4 AI 审核功能测试

AI 审核功能是本系统区别于普通视频管理系统的重要部分。该功能主要用于在人工审核前对视频内容进行初步分析, 辅助管理员判断视频是否存在违规风险。测试时重点验证审核任务创建、消息投递、AI 服务处理、结果回写和后台展示是否正

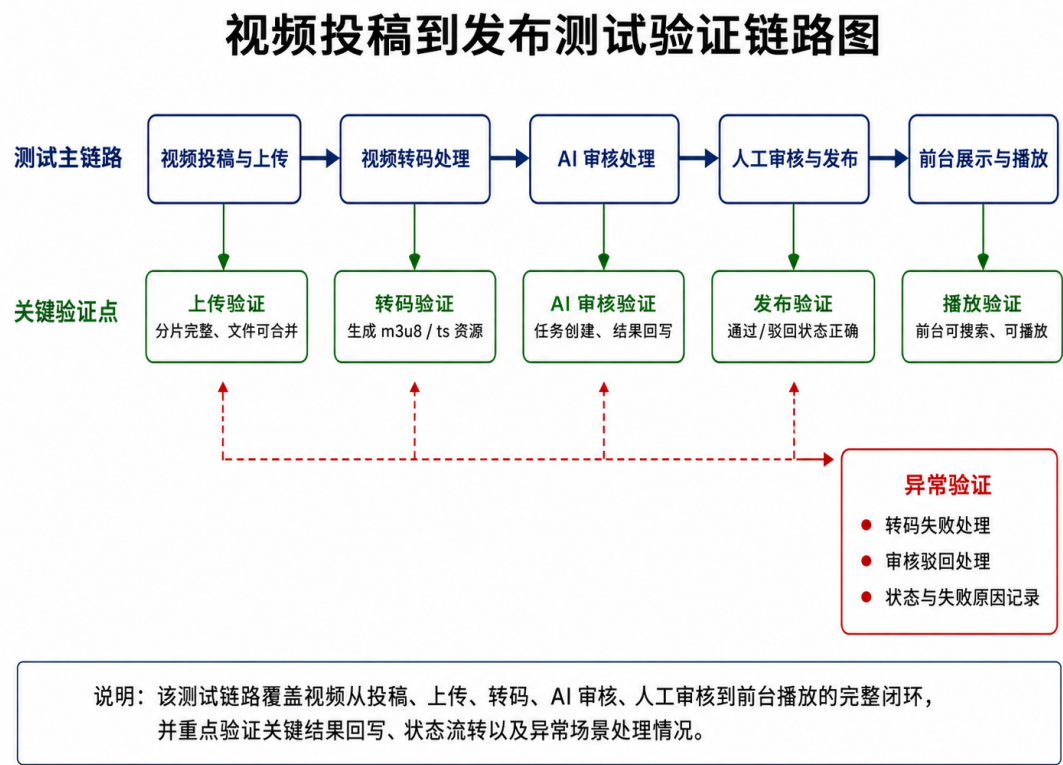


图 6-1 视频投稿到发布测试链路图

表 6-6 视频转码与播放资源测试结果

测试项	测试操作	实际结果	结论
转码任务创建	投稿提交后检查任务状态	系统生成转码任务并记录处理状态	通过
文件合并	分片上传完成后检查合并文件	完整视频文件生成成功，文件大小正确	通过
HLS 生成	调用 FFmpeg 进行转码切片	输出 m3u8 索引文件和 ts 分片文件	通过
播放地址	打开视频详情页获取播放资源	播放器能够加载 HLS 地址	通过
播放统计	播放视频并刷新统计数据	播放量能够异步更新	通过
异常处理	使用异常格式或损坏视频测试	系统记录失败状态并避免发布异常视频	通过

常。

6.4.1 AI 审核任务流转测试

AI 审核任务流转涉及 Java 后端服务、RabbitMQ 消息队列和 Python AI 审核服务。测试时在视频转码完成后检查是否创建审核任务，并观察任务是否被 AI 服务消费，最后检查审核结果是否能够回写到数据库并展示在后台页面。

表 6-7 给出了 AI 审核任务流转测试结果。

表 6-7 AI 审核任务流转测试结果

测试项	测试操作	实际结果	结论
任务创建	视频转码完成后检查审核任务	系统生成 AI 审核任务和分 P 明细记录	通过
消息投递	检查 RabbitMQ 审核队列	审核请求消息能够被正常发送和消费	通过
服务处理	启动 Python AI 审核服务处理任务	服务能够接收请求并返回审核结果	通过
结果回写	检查审核结果入库情况	风险等级、审核建议和原因能够保存	通过
后台展示	打开后台审核详情页	管理员可以查看 AI 审核摘要和风险提示	通过
人工复核	管理员根据结果执行通过或驳回	人工审核结果优先决定最终发布状态	通过

测试结果表明，AI 审核功能能够完成从任务创建到结果展示的闭环。AI 审核结果不会直接替代管理员决策，而是作为人工审核的辅助依据，这与系统设计中“AI 初审 + 人工终审”的内容安全策略一致。

6.4.2 多模态审核结果测试

系统中的 AI 审核服务需要对视频画面、音频和文本信息进行综合分析。测试时选取正常视频和包含敏感提示的视频样例，检查系统是否能够生成审核建议，并在后台页面展示风险来源。

表 6-8 展示了多模态审核结果测试情况。

从测试结果可以看出，多模态审核能够对视频内容提供较为全面的初步判断。不过，AI 审核结果仍可能受到模型能力、样本质量和识别上下文的影响，因此系统保留人工终审环节，以降低误判对最终发布结果的影响。

表 6-8 多模态审核结果测试情况

审核维度	测试内容	实际结果	结论
画面审核	抽取视频关键帧进行识别	能够返回画面风险描述和建议	通过
音频审核	提取音频并进行语音或声音分析	能够返回音频相关审核结果	通过
文本审核	对标题、简介、标签和识别文本进行检查	能够识别明显异常文本信息	通过
综合判断	汇总不同模态的审核结果	能够形成风险等级和审核建议	通过
后台展示	管理端查看审核详情	能够显示风险来源、建议和结果摘要	通过

6.5 跨服务事务一致性测试

跨服务事务测试主要验证系统在远程调用失败、数据库写入异常或业务校验失败时，是否能够保持关键业务数据状态一致。由于系统引入 Seata 分布式事务组件，测试时重点关注互动计数、用户资产变更、投稿审核状态更新等涉及多个服务协作的写操作^[19]。

表 6-9 跨服务事务一致性测试用例

测试项	测试操作	实际结果	结论
投币事务	模拟投币过程中用户资产更新异常	互动记录和用户资产未出现单边提交	通过
审核事务	模拟审核发布过程中状态迁移失败	投稿状态和正式视频数据保持一致	通过
远程调用失败	临时关闭被调用服务后执行跨服务写操作	系统返回异常,相关数据未被错误提交	通过

测试结果表明，系统在涉及多个服务的关键写操作中能够通过全局事务或异常回滚机制降低数据不一致风险。对于播放计数、搜索索引更新等非强一致场景，系统仍采用异步补偿方式进行处理。

6.6 异常场景与安全性测试

异常场景测试主要用于验证系统在非正常输入、服务不可用、权限不足和重复请求等情况下是否具备基本的容错能力。相比正常流程测试，异常测试更能反映系统稳定性和安全性。

表 6-10 展示了系统异常场景测试结果。

表 6-10 异常场景测试结果

异常类型	测试操作	实际结果	结论
未登录访问	游客执行评论、点赞、投稿等操作	系统返回登录提示或拒绝请求	通过
权限不足	普通用户访问后台管理接口	请求被网关或后台服务拦截	通过
验证码错误	输入错误验证码进行登录	登录失败并返回错误提示	通过
重复提交	重复上传同一分片或重复提交审核操作	系统能够避免重复处理造成状态异常	通过
文件异常	上传损坏视频或不支持格式文件	转码失败状态被记录，视频不会被发布	通过
AI 服务异常	停止 AI 服务后触发审核任务	系统保留任务状态，便于后续重试或人工处理	通过
搜索异常	Elasticsearch 不可用时执行搜索	系统返回异常提示，不影响其他模块运行	通过

测试结果表明，系统在常见异常场景下能够进行基本拦截和状态记录。对于依赖外部组件的功能，系统仍然需要进一步完善重试机制和告警机制，以提高生产环境下的可维护性。

6.7 测试结果分析

6.7.1 测试结果汇总

综合上述测试结果，系统的主要功能模块均能够按照预期运行。表 6-11 对测试结果进行汇总。

由表 6-11 可以看出，系统基本完成了需求分析中提出的核心功能。用户端能够支持视频浏览和互动，创作中心能够完成投稿流程，后台管理端能够完成内容审核，资源服务能够完成视频转码和播放资源生成，AI 审核服务能够为管理员提供审核辅助信息。

6.7.2 存在问题与改进方向

虽然系统通过了主要功能测试，但测试过程中也暴露出一些需要后续改进的问题。

第一，系统测试主要在本地或小规模环境下完成，尚未进行大规模并发压力测

表 6-11 系统测试结果汇总

测试模块	测试内容	实际结果	结论
用户端功能	浏览、搜索、播放、评论、弹幕和互动	页面展示正常，交互结果能够保存和更新	通过
创作中心	封面上传、分片上传、投稿提交和状态查看	投稿流程完整，状态流转正确	通过
后台管理	分类管理、用户管理、视频审核和权限控制	管理功能可用，普通用户无法越权访问	通过
视频处理	分片合并、视频转码、HLS 文件生成和播放	视频能够转码并在前台播放	通过
AI 审核	任务创建、消息投递、结果回写和后台展示	AI 审核结果能够辅助人工审核	通过
异常处理	未登录、权限不足、文件异常和服务异常	系统能够拦截或记录异常状态	通过

试。因此，系统在高并发上传、高频播放统计和大量弹幕发送场景下的性能表现仍需要进一步验证。

第二，视频文件存储主要采用本地文件目录方式，能够满足开发和演示需求，但在生产环境中存在扩展性不足的问题。后续可以引入对象存储或分布式文件系统，提高视频资源存储和访问能力。

第三，AI 审核结果可以辅助管理员判断风险，但模型识别准确率仍可能受到样本内容、音频质量、画面清晰度和上下文语义的影响。因此，系统需要继续保留人工终审机制，并在后续增加更多审核样本以优化审核效果。

第四，当前测试主要以手工功能测试为主，自动化测试覆盖不足。后续可以补充接口自动化测试、单元测试和持续集成流程，以提高系统长期维护过程中的稳定性。

6.8 本章小结

本章对在线视频分享平台进行了系统测试与结果分析。首先介绍了系统测试环境和测试方案，然后分别对用户端功能、创作中心功能、后台管理功能、视频处理流程、AI 审核流程、跨服务事务一致性和异常场景进行了测试。测试结果表明，系统能够完成视频上传、转码、AI 初审、人工审核、前台发布和用户互动等核心功能，整体业务流程能够形成闭环。

同时，测试也说明系统在大规模并发测试、文件存储扩展、AI 审核准确率和自

动化测试方面仍有进一步优化空间。后续可以从性能压测、分布式存储、模型优化和自动化测试体系建设等方面继续完善系统。

第 7 章 总结与展望

1. 把第7章与后续结论合并为一章：结论
2. 内容简略一些，写清楚两方面内容：系统特点及创新点；存在的不足及今后可行的解决方案。

7.1 研究工作总结

本文围绕“基于微服务架构的在线视频分享平台设计与实现”展开研究与开发，完成了一个具有视频浏览、视频投稿、分片上传、转码播放、用户互动、后台管理和 AI 辅助审核能力的在线视频分享平台原型。系统采用前后端分离架构，后端按照业务能力划分为网关服务、主站服务、资源服务、互动服务、后台服务和公共基础模块，AI 审核服务则以独立服务形式运行。

在需求分析和总体设计阶段，本文明确了游客、注册用户、内容创作者、管理员和 AI 审核服务的功能边界，分析了系统的功能需求、非功能需求和可行性。随后，本文设计了系统总体架构、服务模块划分、网关路由、数据库结构、缓存搜索、文件存储、消息通信和视频状态流转流程，使系统具备较清晰的模块边界和业务闭环。

在系统实现阶段，本文重点完成了视频生命周期相关功能。创作者可以在创作中心提交视频和封面，系统通过分片上传降低大文件上传失败的影响；资源服务负责视频合并、转码和 HLS 播放资源生成；后台服务负责分类管理和人工审核；AI 审核服务负责对视频语音、声音事件和关键帧内容进行分析，并将风险等级、风险标签和审核建议返回后端。管理员最终结合 AI 审核结果进行人工通过或驳回，从而形成“AI 初审 + 人工终审”的协同审核机制。

在系统测试阶段，本文围绕用户端、创作中心、资源处理、后台管理、AI 审核和异常场景设计了测试用例。测试内容覆盖登录、视频浏览、视频播放、评论弹幕、点赞收藏投币、封面上传、视频分片上传、视频转码、审核任务创建、AI 结果回写和后台人工审核等功能。测试结果表明，系统能够完成在线视频分享平台的主要业务流程，整体功能达到预期目标。

7.2 项目主要特点

本项目的主要特点可以概括为三个方面。第一，系统围绕视频生命周期构建完整流程，而不是只实现简单的视频播放功能。视频从投稿、上传、转码、审核、发布到播放互动均有相应模块支撑，能够体现视频平台的核心业务链路。

第二，系统将微服务架构与视频资源处理相结合。主站服务、资源服务、互动

服务、后台服务和 AI 服务之间职责相对清晰。资源服务专门处理文件上传和视频转码，互动服务专门处理评论、弹幕和用户行为，后台服务负责分类管理和人工审核，AI 服务负责内容审核辅助分析。这种拆分方式降低了不同业务之间的耦合度。

第三，系统引入了 AI 辅助审核流程。AI 审核服务通过消息队列消费审核任务，对视频语音、声音事件和画面内容进行综合分析，并返回审核建议。管理员结合 AI 结果进行最终判断，使系统在内容治理方面具备一定智能化能力。多模态理解模型的发展也为视频内容审核提供了更丰富的技术基础^[6]。

项目主要特点如表 7-1 所示。

表 7-1 项目主要特点

特点	具体说明
完整视频生命周期	实现视频从投稿、上传、转码、审核、发布到播放互动的闭环流程
微服务业务拆分	将主站、资源、互动、后台和 AI 审核能力拆分为相对独立模块
AI 辅助审核	结合语音识别、声音事件识别和多模态理解辅助管理员审核
人机协同机制	AI 负责初审提示，管理员负责最终审核，兼顾效率和准确性

7.3 项目不足

虽然本系统已经完成了在线视频分享平台的核心功能，但由于时间、硬件环境和项目规模限制，仍然存在一些不足。

第一，系统目前更偏向毕业设计原型，尚未进行大规模并发测试。在真实生产环境中，视频平台会面对大量用户访问、视频上传和播放请求，系统还需要进一步进行性能压测和容量规划。

第二，文件存储方式仍以本地目录为主。该方式适合本地开发和演示，但如果部署到多实例环境中，就会出现文件共享和访问路径管理问题。后续可以考虑接入对象存储或分布式文件系统，提高文件存储的扩展性。

第三，AI 审核准确率还需要进一步验证。当前 AI 审核流程能够输出风险提示和审核建议，但模型结果可能受到视频内容、模型能力和提示词设计影响。对于复杂语义、隐晦表达或低质量视频，AI 仍可能出现误判。因此，当前 AI 审核更适合作为辅助工具，而不是完全自动化审核。

第四，系统自动化测试覆盖还不够完整。当前测试主要围绕核心功能和主要流

程展开，尚未形成完整的单元测试、接口自动化测试和持续集成流程。软件质量模型强调可靠性和可维护性，自动化测试是后续提升系统质量的重要方向^[34]。

系统不足如表 7-2 所示。

表 7-2 系统不足分析

不足类别	具体表现	影响
并发测试不足	未进行大规模用户访问压测	难以评估生产环境承载能力
文件存储限制	主要使用本地文件目录	多实例部署时文件共享不方便
AI 准确率不足	模型误判和漏判仍可能存在	审核结果仍需人工确认
自动化测试不足	缺少完整接口测试和持续集成	后续迭代回归成本较高
推荐能力不足	当前更偏向搜索和基础展示	个性化内容分发能力有限

7.4 后续优化方向

针对当前系统存在的不足，后续可以从以下几个方面进行优化。

第一，优化视频资源存储方式。可以将本地文件存储迁移到对象存储或分布式文件系统，并结合 CDN 提高视频访问速度。这样可以解决多实例部署时文件共享不便的问题，也能提高视频加载效率。

第二，提升系统性能和并发能力。后续可以对首页列表、视频播放、评论弹幕、搜索接口和上传接口进行压力测试，并根据测试结果优化缓存策略、数据库索引和服务扩容方式。对于播放量、点赞数等高频统计数据，可以进一步使用异步队列和批量更新减少数据库压力。

第三，完善 AI 审核能力。后续可以引入更完善的审核规则体系，结合模型结果、关键词规则和人工反馈持续优化审核效果。也可以对不同类型风险建立更细的标签体系，例如暴力、低俗、敏感语音和异常声音等，让审核结果更容易理解。算法内容审核研究也指出，自动化审核需要与人工治理和规则体系结合，才能更好地服务平台管理^[30]。

第四，增强平台推荐和数据分析能力。当前系统主要实现视频浏览和搜索，后续可以基于用户观看、点赞、收藏、投币和关注等行为数据，构建简单推荐策略，让用户更容易发现感兴趣的内容。

第五，完善测试和部署体系。后续可以补充单元测试、接口自动化测试和端到

端测试，并结合持续集成工具进行自动构建和部署。对于微服务系统，还可以增加服务监控、日志追踪和异常告警，提高系统运行可观察性。

后续优化方向如表 7-3 所示。

表 7-3 后续优化方向

优化方向	优化内容	预期效果
文件存储优化	接入对象存储、分布式文件系统或 CDN	提高视频访问和部署扩展能力
性能优化	压力测试、缓存优化、索引优化、服务扩容	提升高并发访问能力
AI 审核优化	完善风险标签、规则体系和人工反馈机制	提高审核准确率和可解释性
推荐能力增强	基于用户行为和分类偏好进行推荐	提升内容分发效果
测试部署完善	增加自动化测试、日志追踪和监报告警	降低后续维护成本

7.5 本章小结

本章对全文工作进行了总结，并分析了项目主要特点、不足和后续优化方向。总体来看，本文完成了一个基于微服务架构的在线视频分享平台原型，实现了视频浏览、视频投稿、分片上传、转码播放、互动行为、后台审核和 AI 辅助审核等功能。系统通过服务拆分、异步任务和人机协同审核机制，形成了较完整的视频内容处理闭环。

同时，系统仍存在并发性能尚未充分验证、文件存储方式偏本地化、AI 审核准确率有待进一步评估、自动化测试体系不够完善等不足。后续可以从对象存储、性能优化、审核规则完善、推荐能力和自动化部署等方面继续改进。通过本课题的设计与实现，可以较完整地理解在线视频分享平台的业务流程和技术结构，也能够体现微服务架构、视频处理、异步任务和智能审核在复杂 Web 系统中的综合应用价值。

结 论

本文围绕“基于微服务架构的在线视频分享平台的设计与实现”展开研究与开发，完成了一个具备视频浏览、视频搜索、在线播放、用户互动、视频投稿、分片上传、视频转码、后台审核和 AI 辅助审核等功能的在线视频分享平台原型。系统以视频内容生命周期为主线，围绕视频从用户投稿到正式发布的完整过程，设计并实现了上传、处理、审核、发布和播放等关键环节，基本满足在线视频分享平台的核心业务需求。

在系统架构方面，本文采用微服务架构对后端功能进行拆分，将网关服务、站业务服务、资源文件服务、互动服务、后台管理服务和公共基础模块分别承担不同职责，降低了系统内部模块耦合度。系统通过 Nacos 实现服务注册与发现，通过 Spring Cloud Gateway 提供统一访问入口，通过 OpenFeign 实现服务间调用，并结合 Redis、Elasticsearch、RabbitMQ 和 Seata 等组件完成缓存、检索、异步任务和跨服务一致性控制。该架构设计使系统具有较清晰的服务边界和较好的扩展基础。

在视频处理方面，本文实现了大文件分片上传、文件合并、FFmpeg 视频转码和 HLS 播放资源生成。该流程能够降低大文件上传失败后的重传成本，并将用户上传的原始视频转换为适合网页端播放的流媒体资源。在内容审核方面，本文设计了“AI 初审 + 人工终审”的协同审核机制。AI 审核服务通过消息队列接收审核任务，对视频语音、声音事件和关键帧内容进行分析，并返回风险等级、风险标签和审核建议。管理员在后台结合 AI 审核结果进行最终判断，从而在提高审核效率的同时保留人工决策能力。

在系统测试方面，本文对用户端功能、创作中心功能、后台管理功能、视频处理流程、AI 审核流程、跨服务事务一致性和异常场景进行了测试。测试结果表明，系统能够完成视频投稿、上传、转码、AI 审核、人工审核、正式发布和前台播放等核心流程，用户互动、搜索检索、后台管理和异常处理等功能也能够正常运行，整体业务链路能够形成闭环。

虽然本文完成了系统核心功能的设计与实现，但系统仍存在一些不足。首先，系统主要在本地和小规模环境下完成测试，尚未进行大规模并发压力测试；其次，视频文件主要采用本地目录存储，在多实例部署和大规模文件管理场景下仍存在扩展

性不足的问题；再次，AI 审核结果受模型能力、视频质量和上下文语义影响，仍可能出现误判或漏判，因此目前更适合作为人工审核的辅助工具；最后，系统自动化测试、日志监控和持续集成能力仍有进一步完善空间。

后续工作可以从以下几个方面继续优化：一是引入对象存储或分布式文件系统，提高视频资源存储和访问能力；二是对首页访问、视频播放、评论弹幕、搜索查询和视频上传等高频场景进行压力测试，并根据测试结果优化缓存策略和服务扩容方案；三是进一步完善 AI 审核规则体系和风险标签体系，结合人工反馈提升审核准确率和可解释性；四是补充接口自动化测试、单元测试、日志追踪和系统监控，提高系统长期运行和维护能力。

综上所述，本文完成了一个基于微服务架构的在线视频分享平台原型，实现了视频处理、内容审核、用户互动和后台管理等核心功能。该系统能够体现微服务架构、异步任务处理、视频转码播放和智能内容审核技术在复杂 Web 平台中的综合应用价值，也为后续进一步扩展为更完善的视频内容服务平台奠定了基础。

参考文献

- [1] Lewis J, Fowler M. Microservices: a Definition of This New Architectural Term[Z]. <https://martinfowler.com/articles/microservices.html>. Accessed: 2026-05-03. 2014.
- [2] Newman S. Building Microservices: Designing Fine-Grained Systems[M]. 2nd ed. Sebastopol: O'Reilly Media, 2021.
- [3] Richardson C. Microservices Patterns: With Examples in Java[M]. Shelter Island: Manning Publications, 2018.
- [4] Radford A, Kim J W, Xu T, et al. Robust Speech Recognition via Large-Scale Weak Supervision[C] //Proceedings of Machine Learning Research: Proceedings of the 40th International Conference on Machine Learning: vol. 202. PMLR, 2023: 28492-28518.
- [5] Gemmeke J F, Ellis D P W, Freedman D, et al. Audio Set: An Ontology and Human-Labeled Dataset for Audio Events[C]//2017 IEEE International Conference on Acoustics, Speech and Signal Processing. IEEE, 2017: 776-780.
- [6] Bai J, Bai S, Yang S, et al. Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond[Z]. <https://arxiv.org/abs/2308.12966>. arXiv:2308.12966. 2023.
- [7] FFmpeg Developers. FFmpeg Documentation[Z]. <https://ffmpeg.org/documentation.html>. Accessed: 2026-05-03. 2026.
- [8] Pantos R, May W. RFC 8216: HTTP Live Streaming[R]. 8216. RFC Editor, 2017.
- [9] VMware Tanzu. Spring Cloud Reference Documentation[Z]. <https://docs.spring.io/spring-cloud/docs/current/reference/html/>. Accessed: 2026-05-03. 2026.
- [10] Nacos. What is Nacos[Z]. <https://nacos.io/en-us/docs/what-is-nacos.html>. Accessed: 2026-05-03. 2026.
- [11] Redis Ltd. Redis Documentation[Z]. <https://redis.io/docs/latest/>. Accessed: 2026-05-03. 2026.
- [12] Elastic. Elasticsearch Reference[Z]. <https://www.elastic.co/docs/reference/elasticsearch>. Accessed: 2026-05-03. 2026.
- [13] Broadcom. RabbitMQ Documentation[Z]. <https://www.rabbitmq.com/docs>. Accessed: 2026-05-03. 2026.
- [14] TensorFlow. Sound Classification with YAMNet[Z]. <https://www.tensorflow.org/hub/tutorials/yamnet>. Accessed: 2026-05-03. 2024.
- [15] Ramírez S. FastAPI Documentation[Z]. <https://fastapi.tiangolo.com/>. Accessed: 2026-05-03. 2026.
- [16] VMware Tanzu. Spring Boot Reference Documentation[Z]. <https://docs.spring.io/spring-boot/index.html>. Accessed: 2026-05-03. 2026.
- [17] VMware Tanzu. Spring Cloud Gateway Reference Documentation[Z]. <https://docs.spring.io/spring-cloud-gateway/reference/index.html>. Accessed: 2026-05-03. 2026.
- [18] VMware Tanzu. Spring Cloud OpenFeign[Z]. <https://spring.io/projects/spring-cloud-openfeign>. Accessed: 2026-05-03. 2026.
- [19] Apache Software Foundation. What Is Seata?[Z]. <https://seata.apache.org/docs/overview/what-is-seata/>. Accessed: 2026-05-03. 2026.
- [20] Oracle. MySQL 8.0 Reference Manual[Z]. <https://dev.mysql.com/doc/refman/8.0/en/>. Accessed:

- 2026-05-03. 2026.
- [21] MyBatis. MyBatis 3 Reference Documentation[Z]. <https://mybatis.org/mybatis-3/>. Accessed: 2026-05-03. 2026.
- [22] WHATWG. HTML Living Standard: The Video Element[Z]. <https://html.spec.whatwg.org/multipage/media.html>. Accessed: 2026-05-03. 2026.
- [23] Vue.js Team. Vue.js Guide[Z]. <https://vuejs.org/guide/introduction>. Accessed: 2026-05-03. 2026.
- [24] Vite Team. Vite Guide[Z]. <https://vite.dev/guide/>. Accessed: 2026-05-03. 2026.
- [25] Vue Router Team. Vue Router Documentation[Z]. <https://router.vuejs.org/>. Accessed: 2026-05-03. 2026.
- [26] Pinia Team. Pinia Documentation[Z]. <https://pinia.vuejs.org/>. Accessed: 2026-05-03. 2026.
- [27] Element Plus Team. Element Plus Documentation[Z]. <https://element-plus.org/>. Accessed: 2026-05-03. 2026.
- [28] Axios Contributors. Axios Documentation[Z]. <https://axios-http.com/docs/intro>. Accessed: 2026-05-03. 2026.
- [29] Kleppmann M. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems[M]. Sebastopol: O'Reilly Media, 2017.
- [30] Gorwa R, Binns R, Katzenbach C. Algorithmic Content Moderation: Technical and Political Challenges in the Automation of Platform Governance[J]. Big Data & Society, 2020, 7(1).
- [31] Bass L, Clements P, Kazman R. Software Architecture in Practice[M]. 4th ed. Boston: Addison-Wesley, 2021.
- [32] VMware Tanzu. Spring Security Reference Documentation[Z]. <https://docs.spring.io/spring-security/reference/index.html>. Accessed: 2026-05-03. 2026.
- [33] Amershi S, Weld D, Vorvoreanu M, et al. Guidelines for Human-AI Interaction[C]//Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems. ACM, 2019: 1-13.
- [34] ISO/IEC. ISO/IEC 25010:2011 Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – System and Software Quality Models[S]. Geneva, 2011.
- [35] Fowler M. Patterns of Enterprise Application Architecture[M]. Boston: Addison-Wesley, 2002.
- [36] Evans E. Domain-Driven Design: Tackling Complexity in the Heart of Software[M]. Boston: Addison-Wesley, 2003.
- [37] Encode. Uvicorn Documentation[Z]. <https://uvicorn.dev/>. Accessed: 2026-05-03. 2026.
- [38] Pika Contributors. Introduction to Pika[Z]. <https://pika.readthedocs.io/>. Accessed: 2026-05-03. 2026.
- [39] Silero Team. Silero VAD: Pre-trained Enterprise-grade Voice Activity Detector[Z]. <https://github.com/snakers4/silero-vad>. Accessed: 2026-05-03. 2026.

附 录

如果附录没有内容，
可以直接删除。

附录相关内容…

附录 A L^AT_EX 环境的安装

L^AT_EX 环境的安装。

附录 B B^IThesis 使用说明

B^IThesis 使用说明。

附录是毕业设计（论文）主体的补充项目，为了体现整篇文章的完整性，写入正文又可能有损于论文的条理性、逻辑性和精炼性，这些材料可以写入附录段，但对于每一篇文章并不是必须的。附录依次用大写正体英文字母 A、B、C……编序号，如附录 A、附录 B。阅后删除此段。

附录正文样式与文章正文相同：宋体、小四；行距：22 磅；间距段前段后均为 0 行。阅后删除此段。

致 谢

值此论文完成之际，首先向我的导师……

致谢正文样式与文章正文相同：宋体、小四；行距：22 磅；间距段前段后均为 0 行。阅后删除此段。